

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería

Sistema de Gestión Académica

Escuela de Educación Básica «Provincias Unidas»

Arquitectura de Microservicios, API REST
y Contenerización con Docker

Versión 2.0 — Entrega 2

Integrante	Rol
Bedón Viteri Keyla Betzabé	Arquitecto
Castro López Pedro Leonardo	Líder de Desarrollo
Emanuel Pino Juliana Romina	Responsable de Documentación
Luna Mora Ernesto Gregory	Responsable de Calidad

El Empalme, Guayas — Ecuador

2026

Índice

1. Resumen Ejecutivo	4
2. Requerimientos Funcionales — Sistema Principal	4
2.1. Gestión de Usuarios	4
2.2. Gestión de Años Lectivos	7
2.3. Gestión de Estudiantes	8
2.4. Gestión de Matrículas	11
2.5. Gestión de Grados y Asignaturas	13
2.6. Gestión de Carga Horaria	15
2.7. Reportes del Sistema Principal	17
3. Requerimientos Funcionales — Microservicio Docente	19
3.1. Gestión de Calificaciones	19
3.2. Gestión de Asistencia	23
3.3. Gestión de Horarios	25
3.4. Reportes del Microservicio Docente	26
4. Requerimientos No Funcionales	28
4.1. Seguridad	28
4.2. Disponibilidad	31
4.3. Usabilidad	32
4.4. Rendimiento	34
4.5. Mantenibilidad	35
4.6. Escalabilidad	36
5. Contrato API REST	36
5.1. URL Base	37
5.2. Sistema Principal (Puerto 8080)	37
5.2.1. Autenticación	37
5.2.2. Usuarios	37
5.2.3. Años Lectivos	37
5.2.4. Estudiantes	38
5.2.5. Matrículas	38
5.2.6. Grados y Asignaturas	38
5.2.7. Asignaciones y Horarios	39
5.2.8. Reportes — Sistema Principal	39
5.3. Microservicio Docente (Puerto 8081)	40
5.3.1. Actividades y Calificaciones	40
5.3.2. Asistencia y Períodos de Evaluación	40
5.3.3. Reportes — Microservicio Docente	41
5.4. Códigos de Respuesta HTTP	41
6. Arquitectura del Sistema	42
6.1. Servicios	42

6.2. Justificación de la Arquitectura	42
7. Marco Normativo	42
8. Registro de Cambios — Entrega 2	43
9. Diagramas C4 de Arquitectura	43
9.1. C4 Nivel 1 — Contexto del Sistema	44
9.2. C4 Nivel 2 — Contenedores	45
9.3. C4 Nivel 3 — Componentes del Sistema Principal	47
9.4. C4 Nivel 3 — Componentes del Microservicio Docente	48
10. Architecture Decision Records (ADR)	48
10.1. Arquitectura general	48
10.2. Base de datos	50
10.3. Seguridad	51
10.4. Contenerización y despliegue	53
10.5. Backend	55
10.6. Frontend	56
10.7. Calidad y proceso	56
11. Modelo de Datos Distribuido	57
11.1. Schema <code>sga_main</code> — Sistema Principal	59
11.2. Schema <code>sga_docente</code> — Microservicio Docente	60
12. Estrategia de Contenerización con Docker	60
12.1. Dockerfile — Sistema Principal	61
12.2. Dockerfile — Microservicio Docente	62
12.3. Dockerfile — Frontend React	62
12.4. <code>docker-compose.yml</code> completo	63
12.5. Variables de entorno — archivo <code>.env</code>	66
12.6. Resumen de la estrategia de contenedores	67
13. Protocolo Experimental Reproducible	67
13.1. Descripción en español	67
13.1.1. Requisitos del entorno	68
13.1.2. Pasos de despliegue	68
13.1.3. Resolución de problemas frecuentes	69
13.2. Reproducible Experimental Protocol (English)	69

Resumen

El Sistema de Gestión Académica (SGA) para la Escuela de Educación Básica «Provincias Unidas» automatiza los procesos de matrícula, registro de calificaciones, control de asistencia y generación de reportes académicos mediante una arquitectura de microservicios desplegada con Docker Compose. Se diseñaron dos microservicios independientes (Sistema Principal en puerto 8080 y Microservicio Docente en puerto 8081) comunicados a través de un API Gateway con autenticación JWT y cifrado TLS 1.3. La base de datos distribuida utiliza PostgreSQL 16 con dos schemas separados (`sga_main` y `sga_docente`). El contrato API REST define 47 endpoints versionados bajo `/api/v1` y `/docente/v1`. La estrategia de contenerización garantiza reproducibilidad del entorno en menos de 30 minutos mediante `docker compose up`. El sistema atiende a 344 estudiantes, 14 docentes y 4 tipos de actores con control de acceso basado en roles (RBAC).

Abstract — The Academic Management System (SGA) for «Provincias Unidas» Basic Education School automates enrollment, grade recording, attendance tracking, and academic report generation through a microservices architecture deployed with Docker Compose. Two independent microservices were designed (Main System on port 8080 and Teacher Microservice on port 8081) communicating through an API Gateway with JWT authentication and TLS 1.3 encryption. The distributed database uses PostgreSQL 16 with two separate schemas (`sga_main` and `sga_docente`). The REST API contract defines 47 versioned endpoints. The containerization strategy guarantees environment reproducibility in under 30 minutes using `docker compose up`.

Palabras clave — microservicios, API REST, Docker, PostgreSQL, gestión académica, JWT, arquitectura distribuida.

Keywords — microservices, REST API, Docker, PostgreSQL, academic management, JWT, distributed architecture.

1. Resumen Ejecutivo

La Escuela de Educación Básica «Provincias Unidas», ubicada en El Empalme, Guayas, gestiona actualmente sus procesos académicos mediante hojas de cálculo compartidas en Google Drive. Este modelo presenta limitaciones críticas: ausencia de control de acceso por roles, dependencia de plataformas externas del Ministerio de Educación que históricamente han sido descontinuadas, y la imposibilidad de garantizar la continuidad y trazabilidad de la información académica a largo plazo.

El presente proyecto propone el diseño e implementación del **Sistema de Gestión Académica (SGA)**, una solución de software distribuido que centraliza y automatiza los procesos de matriculación, registro de calificaciones, control de asistencia y generación de reportes académicos de la institución.

El sistema será utilizado por cuatro tipos de actores: el **Director**, la **Secretaría**, los **Docentes** y el personal de **Soporte Técnico**. La institución cuenta con 344 estudiantes distribuidos desde Inicial 1 hasta Décimo año de Educación General Básica, 14 docentes y un esquema de calificación mixto conforme a los lineamientos del Ministerio de Educación del Ecuador [1].

La arquitectura de microservicios se justifica por tres razones fundamentales. Primero, la institución opera con dos dominios de negocio claramente diferenciados [2]. Segundo, la disponibilidad diferenciada es un requisito real: si el módulo docente presenta una falla, el sistema administrativo debe continuar operando sin interrupción [3]. Tercero, la arquitectura permite incorporar futuros módulos sin afectar los servicios existentes [4].

2. Requerimientos Funcionales — Sistema Principal

Los requerimientos funcionales se identificaron mediante entrevistas con los actores clave de la institución y análisis de los procesos académicos vigentes, siguiendo las prácticas recomendadas de ingeniería de requerimientos [5, 6].

2.1 Gestión de Usuarios

Tabla 1 — Requisitos funcionales: Creación y gestión de cuentas

Campo	Contenido
ID	RF-01
Nombre	Gestión de cuentas de usuario
Descripción	El sistema debe permitir crear, editar, activar y desactivar las cuentas de los usuarios registrados en la plataforma.
Entradas	Datos del usuario: nombre completo, correo electrónico, rol asignado y estado inicial de la cuenta.
Salidas	Cuenta de usuario creada, actualizada o con estado modificado según la acción ejecutada.
Actor	Director, Soporte Técnico.
Prioridad	Alta.
Criterio de verificación	El sistema crea, edita o cambia el estado de la cuenta correctamente y refleja los cambios de forma inmediata en la lista de usuarios; un usuario desactivado no puede iniciar sesión.

Elaborado por: Equipo BCEL.

Tabla 2 — Requisitos funcionales: Asignación de roles

Campo	Contenido
ID	RF-02
Nombre	Asignación de roles
Descripción	El sistema debe permitir asignar a cada usuario uno de los roles definidos: Director, Secretaría, Docente o Soporte Técnico, determinando así sus permisos de acceso.
Entradas	Identificador del usuario y rol seleccionado.
Salidas	Usuario con rol asignado; el menú y las funcionalidades disponibles se ajustan según el rol.
Actor	Director, Soporte Técnico.
Prioridad	Alta.
Criterio de verificación	Al iniciar sesión, el usuario visualiza únicamente las opciones correspondientes a su rol; los accesos no autorizados retornan el código HTTP 403.

Elaborado por: Equipo BCEL.

Tabla 3 — Requisitos funcionales: Restablecimiento de contraseña

Campo	Contenido
ID	RF-03
Nombre	Restablecimiento de contraseña
Descripción	El sistema debe permitir al administrador restablecer la contraseña de cualquier usuario.
Entradas	Identificador del usuario cuya contraseña se restablecerá.
Salidas	Contraseña provisional generada y enviada al correo del usuario; la cuenta queda habilitada.
Actor	Director, Soporte Técnico.
Prioridad	Media.
Criterio de verificación	El usuario recibe la nueva contraseña provisional y al intentar ingresar se le solicita obligatoriamente su cambio.

Elaborado por: Equipo BCEL.

Tabla 4 — Requisitos funcionales: Cambio obligatorio en primer ingreso

Campo	Contenido
ID	RF-04
Nombre	Cambio obligatorio de contraseña
Descripción	El sistema debe forzar al usuario a cambiar su contraseña provisional en el primer inicio de sesión.
Entradas	Contraseña provisional y nueva contraseña elegida por el usuario (con su confirmación).
Salidas	Contraseña actualizada; el usuario es redirigido al panel principal del sistema.
Actor	Todos los usuarios.
Prioridad	Alta.
Criterio de verificación	El usuario no puede acceder a ninguna funcionalidad del sistema hasta completar el cambio de contraseña.

Elaborado por: Equipo BCEL.

Tabla 5 — Requisitos funcionales: Bloqueo por intentos fallidos

Campo	Contenido
ID	RF-05
Nombre	Bloqueo de cuenta por intentos fallidos
Descripción	El sistema debe bloquear automáticamente la cuenta de un usuario tras un número definido de intentos fallidos de inicio de sesión.
Entradas	Credenciales incorrectas introducidas consecutivamente por el usuario.
Salidas	Cuenta bloqueada temporalmente; el sistema muestra un mensaje de error e impide nuevos intentos.
Actor	Todos los usuarios.
Prioridad	Alta.
Criterio de verificación	Tras el número configurado de intentos fallidos, la cuenta queda bloqueada y solo el Director o Soporte Técnico puede desbloquearla; un atacante no puede continuar intentando credenciales.

Elaborado por: Equipo BCEL.

2.2 Gestión de Años Lectivos

Tabla 6 — Requisitos funcionales: Gestión de períodos académicos

Campo	Contenido
ID	RF-06
Nombre	Creación y gestión de períodos académicos
Descripción	El sistema debe permitir crear y administrar períodos académicos anuales con sus fechas de inicio y fin.
Entradas	Nombre del año lectivo, fecha de inicio y fecha de fin.
Salidas	Año lectivo creado o actualizado y disponible para asociar matrículas y calificaciones.
Actor	Director.
Prioridad	Alta.
Criterio de verificación	El año lectivo aparece en el listado del sistema y puede seleccionarse al realizar matrículas u otras operaciones.

Elaborado por: Equipo BCEL.

Tabla 7 — Requisitos funcionales: Año lectivo activo

Campo	Contenido
ID	RF-07
Nombre	Establecer año lectivo activo
Descripción	El sistema debe permitir marcar un único año lectivo como activo, sirviendo de referencia para todas las operaciones del sistema.
Entradas	Identificador del año lectivo a activar.
Salidas	Año lectivo marcado como activo; los registros previos quedan en estado histórico.
Actor	Director.
Prioridad	Alta.
Criterio de verificación	Solo existe un año lectivo activo a la vez; al activar uno nuevo, el anterior cambia automáticamente a estado histórico.

Elaborado por: Equipo BCEL.

Tabla 8 — Requisitos funcionales: Filtro por año lectivo

Campo	Contenido
ID	RF-08
Nombre	Filtrado de información por año lectivo
Descripción	Toda la información del sistema (estudiantes, matrículas, calificaciones) debe filtrarse por el año lectivo activo.
Entradas	Año lectivo activo seleccionado en el sistema.
Salidas	Datos mostrados correspondientes exclusivamente al año lectivo activo.
Actor	Todos los usuarios.
Prioridad	Alta.
Criterio de verificación	Al consultar cualquier módulo del sistema, los datos reflejados corresponden al año lectivo activo; el usuario puede cambiar al modo histórico seleccionando un año anterior.

Elaborado por: Equipo BCEL.

2.3 Gestión de Estudiantes

Tabla 9 — Requisitos funcionales: Registro manual de estudiantes

Campo	Contenido
ID	RF-09
Nombre	Registro manual de estudiantes
Descripción	El Director debe poder registrar manualmente a los estudiantes a partir del listado descargado del sistema CAS del Ministerio de Educación.
Entradas	Listado exportado del sistema CAS con los datos de los estudiantes.
Salidas	Estudiantes registrados en el sistema con sus datos básicos importados.
Actor	Director.
Prioridad	Alta.
Criterio de verificación	Los estudiantes registrados aparecen en el módulo de estudiantes y pueden ser matriculados en un grado.

Elaborado por: Equipo BCEL.

Tabla 10 — Requisitos funcionales: Datos personales del estudiante

Campo	Contenido
ID	RF-10
Nombre	Registro de datos personales
Descripción	El sistema debe permitir registrar los datos personales del estudiante: nombres, apellidos, número de cédula, fecha de nacimiento, género, dirección, teléfono y correo electrónico.
Entradas	Formulario con los datos personales del estudiante.
Salidas	Ficha del estudiante creada o actualizada con sus datos personales completos.
Actor	Director, Secretaría.
Prioridad	Alta.
Criterio de verificación	El sistema valida el formato de la cédula y el correo electrónico; los campos obligatorios no pueden quedar vacíos.

Elaborado por: Equipo BCEL.

Tabla 11 — Requisitos funcionales: Registro de discapacidad

Campo	Contenido
ID	RF-11
Nombre	Registro de condición de discapacidad
Descripción	El sistema debe permitir registrar si el estudiante tiene alguna discapacidad y, en caso afirmativo, el porcentaje correspondiente según el carné del CONADIS.
Entradas	Indicador de discapacidad (sí/no) y porcentaje de discapacidad si aplica.
Salidas	Información de discapacidad registrada en la ficha del estudiante.
Actor	Director, Secretaría.
Prioridad	Media.
Criterio de verificación	El campo de porcentaje solo se habilita si se indica que el estudiante tiene discapacidad; el sistema almacena y muestra el dato correctamente en reportes.

Elaborado por: Equipo BCEL.

Tabla 12 — Requisitos funcionales: Datos del representante legal

Campo	Contenido
ID	RF-12
Nombre	Registro del representante legal
Descripción	El sistema debe permitir registrar los datos del representante legal del estudiante: nombres completos, parentesco, número de teléfono y correo electrónico.
Entradas	Formulario con los datos del representante legal.
Salidas	Datos del representante asociados al perfil del estudiante.
Actor	Director, Secretaría.
Prioridad	Alta.
Criterio de verificación	La ficha del estudiante muestra correctamente los datos del representante; el sistema valida el formato del teléfono y el correo.

Elaborado por: Equipo BCEL.

Tabla 13 — Requisitos funcionales: Datos médicos básicos

Campo	Contenido
ID	RF-13
Nombre	Registro de datos médicos básicos
Descripción	El sistema debe permitir registrar información médica básica del estudiante, como tipo de sangre, alergias conocidas y condiciones de salud relevantes.
Entradas	Formulario de datos médicos básicos del estudiante.
Salidas	Información médica registrada en la ficha del estudiante y disponible para consulta.
Actor	Director, Secretaría.
Prioridad	Media.
Criterio de verificación	Los datos médicos se almacenan correctamente y son accesibles desde la ficha del estudiante por los roles autorizados.

Elaborado por: Equipo BCEL.

2.4 Gestión de Matrículas

Tabla 14 — Requisitos funcionales: Matriculación de estudiantes

Campo	Contenido
ID	RF-14
Nombre	Matriculación en grado y año lectivo
Descripción	El sistema debe permitir matricular a un estudiante en un grado y año lectivo determinado.
Entradas	Identificador del estudiante, grado seleccionado y año lectivo activo.
Salidas	Matrícula registrada en el sistema con estado activo y asociada al grado y año lectivo correspondiente.
Actor	Director, Secretaría.
Prioridad	Alta.
Criterio de verificación	La matrícula queda registrada; el estudiante aparece en el listado del grado correspondiente y puede recibir calificaciones y asistencia.

Elaborado por: Equipo BCEL.

Tabla 15 — Requisitos funcionales: Validación de matrículas duplicadas

Campo	Contenido
ID	RF-15
Nombre	Validación de matrículas duplicadas
Descripción	El sistema debe validar que un estudiante no pueda estar matriculado más de una vez en el mismo año lectivo.
Entradas	Intento de creación de una segunda matrícula para el mismo estudiante en el mismo año lectivo.
Salidas	Mensaje de error y rechazo de la operación; la matrícula duplicada no se registra.
Actor	Director, Secretaría.
Prioridad	Alta.
Criterio de verificación	El sistema retorna el código HTTP 409 y muestra un mensaje claro indicando que el estudiante ya posee una matrícula en el año lectivo activo.

Elaborado por: Equipo BCEL.

Tabla 16 — Requisitos funcionales: Cambio de estado de matrícula

Campo	Contenido
ID	RF-16
Nombre	Cambio de estado de matrícula
Descripción	El sistema debe permitir cambiar el estado de una matrícula entre los valores: activa, retirada, promovida o reprobada.
Entradas	Identificador de la matrícula y nuevo estado seleccionado.
Salidas	Estado de la matrícula actualizado; el registro histórico es preservado.
Actor	Director, Secretaría.
Prioridad	Alta.
Criterio de verificación	El nuevo estado se refleja inmediatamente en el listado de matrículas y en los reportes del año lectivo correspondiente.

Elaborado por: Equipo BCEL.

Tabla 17 — Requisitos funcionales: Matrícula automática

Campo	Contenido
ID	RF-17
Nombre	Matrícula automática al siguiente grado
Descripción	El sistema debe generar automáticamente la matrícula del siguiente año lectivo para los estudiantes con estado promovido al cierre del año lectivo actual.
Entradas	Listado de estudiantes con estado «promovido» y nuevo año lectivo activado.
Salidas	Matrículas del nuevo año lectivo creadas automáticamente para los estudiantes promovidos.
Actor	Director (activa el proceso).
Prioridad	Media.
Criterio de verificación	Al activar el nuevo año lectivo, los estudiantes promovidos aparecen matriculados en el grado inmediatamente superior sin necesidad de registro manual.

Elaborado por: Equipo BCEL.

2.5 Gestión de Grados y Asignaturas

Tabla 18 — Requisitos funcionales: Catálogo de grados

Campo	Contenido
ID	RF-18
Nombre	Gestión del catálogo de grados
Descripción	El sistema debe permitir gestionar los grados desde Inicial 1 hasta Décimo año de Educación General Básica.
Entradas	Nombre del grado, nivel educativo y estado (activo/inactivo).
Salidas	Grado creado, actualizado o desactivado según la acción ejecutada.
Actor	Director.
Prioridad	Alta.
Criterio de verificación	Los grados activos están disponibles para la matrícula de estudiantes y la asignación de docentes.

Elaborado por: Equipo BCEL.

Tabla 19 — Requisitos funcionales: Gestión de paralelos

Campo	Contenido
ID	RF-19
Nombre	Gestión de paralelos por grado
Descripción	El sistema debe permitir gestionar los paralelos asociados a cada grado (A, B, C, etc.).
Entradas	Grado al que pertenece el paralelo y letra identificadora del paralelo.
Salidas	Paralelo creado y asociado al grado correspondiente.
Actor	Director.
Prioridad	Alta.
Criterio de verificación	Al matricular un estudiante, el sistema permite seleccionar el paralelo disponible dentro del grado seleccionado.

Elaborado por: Equipo BCEL.

Tabla 20 — Requisitos funcionales: Catálogo de asignaturas

Campo	Contenido
ID	RF-20
Nombre	Gestión del catálogo de asignaturas
Descripción	El sistema debe permitir gestionar el catálogo de asignaturas por nivel educativo, indicando su tipo de escala de calificación.
Entradas	Nombre de la asignatura, nivel educativo al que pertenece y tipo de escala (cualitativa, cuantitativa o mixta).
Salidas	Asignatura registrada en el catálogo y disponible para asignación a docentes.
Actor	Director.
Prioridad	Alta.
Criterio de verificación	Las asignaturas del catálogo se muestran correctamente filtradas por nivel educativo al asignarlas a docentes.

Elaborado por: Equipo BCEL.

Tabla 21 — Requisitos funcionales: Asignación docente a asignaturas

Campo	Contenido
ID	RF-21
Nombre	Asignación de docentes a asignaturas
Descripción	El sistema debe permitir asignar docentes a asignaturas por grado y año lectivo, respetando las modalidades de asignación establecidas.
Entradas	Docente seleccionado, asignatura, grado y año lectivo.
Salidas	Asignación registrada; el docente visualiza los cursos asignados en el microservicio docente.
Actor	Director.
Prioridad	Alta.
Criterio de verificación	El docente asignado puede acceder al registro de calificaciones y asistencia del curso correspondiente desde el módulo docente.

Elaborado por: Equipo BCEL.

2.6 Gestión de Carga Horaria

Tabla 22 — Requisitos funcionales: Modalidades de asignación docente

Campo	Contenido
ID	RF-22
Nombre	Modalidades de asignación docente por nivel
Descripción	El sistema debe gestionar dos modalidades de asignación según el nivel educativo: (a) Inicial 1, Inicial 2 y Preparatoria hasta 4to: un docente titular para todas las materias excepto Inglés y Educación Física; (b) 5to a 10mo: docentes especializados rotativos por asignatura y hora.
Entradas	Nivel educativo del grado y modalidad de asignación configurada.
Salidas	El sistema aplica automáticamente las restricciones de asignación según el nivel del grado seleccionado.
Actor	Director.
Prioridad	Alta.
Criterio de verificación	El sistema no permite asignar Inglés ni Educación Física al docente titular de Inicial 1 e Inicial 2; para 5to–10mo, habilita la selección individual de docente por asignatura y hora.

Elaborado por: Equipo BCEL.

Tabla 23 — Requisitos funcionales: Asignación del Director

Campo	Contenido
ID	RF-23
Nombre	Asignación de docentes por el Director
Descripción	El sistema debe permitir al Director asignar el docente titular y los docentes especializados para cada grado y año lectivo.
Entradas	Grado, año lectivo, docente titular y docentes especializados por asignatura.
Salidas	Asignaciones registradas y visibles en el módulo de carga horaria.
Actor	Director.
Prioridad	Alta.
Criterio de verificación	Todos los grados del año lectivo activo cuentan con al menos un docente asignado; las asignaciones incompletas son señaladas por el sistema con una alerta visual.

Elaborado por: Equipo BCEL.

Tabla 24 — Requisitos funcionales: Construcción del horario anual

Campo	Contenido
ID	RF-24
Nombre	Construcción del horario anual por curso
Descripción	El sistema debe permitir al Director construir el horario anual fijo de cada curso, asignando asignaturas y docentes a cada período y día de la semana.
Entradas	Grado, paralelo, día de la semana, período horario, asignatura y docente.
Salidas	Horario anual del curso registrado y disponible para consulta por docentes y secretaría.
Actor	Director.
Prioridad	Alta.
Criterio de verificación	El horario generado es visualizado correctamente por todos los actores autorizados; el sistema alerta si existe solapamiento de docente en el mismo período.

Elaborado por: Equipo BCEL.

Tabla 25 — Requisitos funcionales: Períodos diarios

Campo	Contenido
ID	RF-25
Nombre	Gestión de períodos horarios diarios
Descripción	El sistema debe gestionar los períodos horarios institucionales diferenciados por nivel: Inicial/1ro-2do tienen receso entre el 3er y 4to período; 3ro-10mo tienen receso después del 4to período.
Entradas	Configuración de períodos institucionales según nivel educativo.
Salidas	Períodos disponibles para la construcción del horario, respetando la estructura institucional establecida.
Actor	Director.
Prioridad	Media.
Criterio de verificación	Los horarios generados reflejan correctamente los períodos y sus horas según el nivel educativo del grado.

Elaborado por: Equipo BCEL.

2.7 Reportes del Sistema Principal

Tabla 26 — Requisitos funcionales: Boletas de calificaciones

Campo	Contenido
ID	RF-26
Nombre	Generación de boletas de calificaciones
Descripción	El sistema debe generar boletas de calificaciones por estudiante y trimestre, mostrando las notas de todas las asignaturas.
Entradas	Identificador de la matrícula del estudiante y trimestre seleccionado.
Salidas	Boleta de calificaciones generada en formato PDF lista para impresión o descarga.
Actor	Director, Secretaría.
Prioridad	Alta.
Criterio de verificación	La boleta generada contiene las calificaciones de todas las asignaturas del estudiante en el trimestre indicado, correctamente formateadas con los datos institucionales.

Elaborado por: Equipo BCEL.

Tabla 27 — Requisitos funcionales: Actas de calificaciones

Campo	Contenido
ID	RF-27
Nombre	Generación de actas de calificaciones
Descripción	El sistema debe generar actas de calificaciones por grado y trimestre, incluyendo las firmas del docente y la directora.
Entradas	Grado, paralelo y trimestre seleccionado.
Salidas	Acta de calificaciones del grado en formato PDF con espacios para firma del docente y la directora.
Actor	Director, Secretaría.
Prioridad	Alta.
Criterio de verificación	El acta contiene las calificaciones de todos los estudiantes matriculados en el grado para el trimestre indicado.

Elaborado por: Equipo BCEL.

Tabla 28 — Requisitos funcionales: Reportes de asistencia general

Campo	Contenido
ID	RF-28
Nombre	Generación de reportes de asistencia general
Descripción	El sistema debe generar reportes de asistencia general por grado, mostrando los totales de presentes, ausentes, justificados y atrasos.
Entradas	Grado y período de tiempo seleccionado.
Salidas	Reporte de asistencia en formato PDF con el resumen por estudiante.
Actor	Director, Secretaría.
Prioridad	Media.
Criterio de verificación	El reporte refleja fielmente los registros de asistencia ingresados por los docentes para el grado y período indicados.

Elaborado por: Equipo BCEL.

Tabla 29 — Requisitos funcionales: Descarga en PDF

Campo	Contenido
ID	RF-29
Nombre	Descarga de reportes en formato PDF
Descripción	Todos los reportes generados por el sistema deben poder descargarse en formato PDF desde el navegador del usuario.
Entradas	Acción de descarga del reporte seleccionado.
Salidas	Archivo PDF generado correctamente y descargado en el dispositivo del usuario.
Actor	Director, Secretaría.
Prioridad	Alta.
Criterio de verificación	El archivo PDF se genera en menos de 3 segundos, es legible, mantiene el formato institucional y puede abrirse en cualquier lector de PDF estándar.

Elaborado por: Equipo BCEL.

3. Requerimientos Funcionales — Microservicio Docente

3.1 Gestión de Calificaciones

Tabla 30 — Requisitos funcionales: Vista restringida del docente

Campo	Contenido
ID	RF-D01
Nombre	Vista restringida del docente
Descripción	El docente debe visualizar únicamente los cursos y asignaturas que le fueron asignados por el Director para el año lectivo activo.
Entradas	Credenciales de inicio de sesión del docente.
Salidas	Panel del docente con exclusivamente sus cursos y asignaturas asignadas.
Actor	Docente.
Prioridad	Alta.
Criterio de verificación	El docente no puede acceder a los cursos o asignaturas de otros docentes; cualquier intento retorna el código HTTP 403.

Elaborado por: Equipo BCEL.

Tabla 31 — Requisitos funcionales: Registro de actividades

Campo	Contenido
ID	RF-D02
Nombre	Registro de actividades evaluativas
Descripción	El docente debe poder registrar actividades evaluativas por asignatura y trimestre con los tipos: Lección Oral, Lección Escrita, Tareas, Talleres, Cuaderno, Trabajo Individual, Exposición.
Entradas	Nombre de la actividad, tipo, asignatura, trimestre y fecha de realización.
Salidas	Actividad evaluativa registrada y disponible para el ingreso de calificaciones por estudiante.
Actor	Docente.
Prioridad	Alta.
Criterio de verificación	La actividad creada aparece en el listado de actividades de la asignatura y trimestre correspondiente; las calificaciones pueden ingresarse inmediatamente.

Elaborado por: Equipo BCEL.

Tabla 32 — Requisitos funcionales: Ingreso de calificaciones

Campo	Contenido
ID	RF-D03
Nombre	Ingreso de calificaciones por actividad
Descripción	El docente debe poder ingresar calificaciones cuantitativas para cada estudiante por actividad evaluativa.
Entradas	Actividad seleccionada, listado de estudiantes del curso y calificación de cada uno.
Salidas	Calificaciones registradas para todos los estudiantes de la actividad seleccionada.
Actor	Docente.
Prioridad	Alta.
Criterio de verificación	Las calificaciones ingresadas se almacenan correctamente y son usadas para el cálculo del promedio formativo.

Elaborado por: Equipo BCEL.

Tabla 33 — Requisitos funcionales: Promedio formativo

Campo	Contenido
ID	RF-D04
Nombre	Cálculo automático del promedio formativo
Descripción	El sistema debe calcular automáticamente el promedio de la evaluación formativa de cada estudiante por asignatura y trimestre, en base a las calificaciones de las actividades registradas.
Entradas	Calificaciones ingresadas de todas las actividades de la asignatura en el trimestre.
Salidas	Promedio formativo calculado y mostrado por asignatura y trimestre.
Actor	Sistema (cálculo automático).
Prioridad	Alta.
Criterio de verificación	El promedio formativo se recalcula automáticamente cada vez que se agrega o edita una calificación; el resultado coincide con el cálculo manual verificado.

Elaborado por: Equipo BCEL.

Tabla 34 — Requisitos funcionales: Promedio trimestral

Campo	Contenido
ID	RF-D05
Nombre	Cálculo del promedio trimestral
Descripción	El sistema debe calcular el promedio trimestral de cada estudiante por asignatura aplicando la ponderación: 70 % evaluación formativa + 30 % evaluación sumativa, conforme a los lineamientos del Ministerio de Educación [1].
Entradas	Promedio formativo y nota de evaluación sumativa del trimestre.
Salidas	Promedio trimestral calculado y disponible para la generación de boletas.
Actor	Sistema (cálculo automático).
Prioridad	Alta.
Criterio de verificación	El promedio trimestral resultante aplica correctamente la fórmula $(0,70 \times P_{formativo}) + (0,30 \times P_{sumativo})$.

Elaborado por: Equipo BCEL.

Tabla 35 — Requisitos funcionales: Escalas de calificación

Campo	Contenido
ID	RF-D06
Nombre	Manejo de escalas de calificación por nivel
Descripción	El sistema debe manejar dos escalas de calificación según el nivel educativo: (a) Inicial, Preparatoria y Básica Elemental: escala cualitativa (A+, A-, B+, B-, C+, C-, D); (b) Básica Media y Superior: escala cuantitativa.
Entradas	Nivel educativo del grado del estudiante.
Salidas	Calificaciones mostradas en la escala correspondiente al nivel educativo del estudiante.
Actor	Sistema.
Prioridad	Alta.
Criterio de verificación	Las boletas de los niveles correspondientes muestran la escala correcta; el sistema no muestra escalas cuantitativas donde corresponde la cualitativa, ni viceversa.

Elaborado por: Equipo BCEL.

Tabla 36 — Requisitos funcionales: Conversión automática de escala

Campo	Contenido
ID	RF-D07
Nombre	Conversión automática de nota a escala cualitativa
Descripción	El sistema debe convertir automáticamente la nota cuantitativa a cualitativa para los niveles que lo requieran, según la tabla de equivalencias del Ministerio de Educación.
Entradas	Nota cuantitativa ingresada y nivel educativo del estudiante.
Salidas	Equivalencia cualitativa calculada y mostrada junto a la nota numérica.
Actor	Sistema.
Prioridad	Alta.
Criterio de verificación	La equivalencia cualitativa generada corresponde exactamente a la tabla oficial del Ministerio de Educación para todos los rangos de nota.

Elaborado por: Equipo BCEL.

Tabla 37 — Requisitos funcionales: Escala mixta

Campo	Contenido
ID	RF-D08
Nombre	Manejo de asignaturas con escala mixta
Descripción	El sistema debe manejar asignaturas con escala mixta como Animación a la Lectura y ECA, que requieren tanto calificación cuantitativa como cualitativa simultáneamente.
Entradas	Calificación cuantitativa de la asignatura con escala mixta.
Salidas	Calificación cuantitativa almacenada y equivalencia cualitativa calculada automáticamente.
Actor	Sistema.
Prioridad	Media.
Criterio de verificación	Las asignaturas de escala mixta muestran ambas representaciones en la boleta del estudiante sin errores de formato.

Elaborado por: Equipo BCEL.

Tabla 38 — Requisitos funcionales: Promedio final anual

Campo	Contenido
ID	RF-D09
Nombre	Cálculo del promedio final anual
Descripción	El sistema debe calcular el promedio final anual de cada estudiante por asignatura, en base a los promedios de los tres trimestres.
Entradas	Promedios trimestrales de los tres trimestres del año lectivo.
Salidas	Promedio final anual calculado por asignatura y estudiante.
Actor	Sistema.
Prioridad	Alta.
Criterio de verificación	El promedio anual es el promedio aritmético de los tres trimestres; el resultado determina si el estudiante es promovido o reprobado en la asignatura.

Elaborado por: Equipo BCEL.

3.2 Gestión de Asistencia

Tabla 39 — Requisitos funcionales: Registro de asistencia diaria

Campo	Contenido
ID	RF-D10
Nombre	Registro de asistencia diaria
Descripción	El docente debe poder registrar la asistencia diaria de cada estudiante del curso asignado.
Entradas	Fecha, listado de estudiantes del curso y estado de asistencia de cada uno.
Salidas	Registros de asistencia almacenados para la fecha seleccionada.
Actor	Docente.
Prioridad	Alta.
Criterio de verificación	El sistema permite registrar la asistencia una vez por día; el registro puede editarse si no ha sido cerrado el período.

Elaborado por: Equipo BCEL.

Tabla 40 — Requisitos funcionales: Estados de asistencia

Campo	Contenido
ID	RF-D11
Nombre	Estados de asistencia
Descripción	El sistema debe registrar los estados de asistencia: Presente, Ausente, Justificado y Atraso para cada estudiante.
Entradas	Estado de asistencia seleccionado por el docente para cada estudiante.
Salidas	Estado registrado correctamente y contabilizado en los totales del trimestre.
Actor	Docente.
Prioridad	Alta.
Criterio de verificación	Cada estado se contabiliza de forma independiente; las ausencias justificadas no se suman a las ausencias sin justificación en los reportes.

Elaborado por: Equipo BCEL.

Tabla 41 — Requisitos funcionales: Totales de asistencia

Campo	Contenido
ID	RF-D12
Nombre	Totalización de asistencias por trimestre
Descripción	El sistema debe totalizar los registros de asistencia por trimestre y por estudiante, mostrando el conteo de cada estado.
Entradas	Registros de asistencia del trimestre para el curso seleccionado.
Salidas	Resumen de asistencia con totales de días presentes, ausentes, justificados y atrasos por estudiante y trimestre.
Actor	Sistema.
Prioridad	Alta.
Criterio de verificación	Los totales calculados coinciden con el conteo manual de los registros del trimestre.

Elaborado por: Equipo BCEL.

3.3 Gestión de Horarios

Tabla 42 — Requisitos funcionales: Consulta de horario personal

Campo	Contenido
ID	RF-D13
Nombre	Consulta del horario personal del docente
Descripción	El docente debe poder consultar su horario personal anual, mostrando todos los cursos, asignaturas, períodos y días que tiene asignados.
Entradas	Sesión activa del docente autenticado.
Salidas	Horario personal del docente mostrado en formato de cuadrícula semanal.
Actor	Docente.
Prioridad	Media.
Criterio de verificación	El horario mostrado coincide con las asignaciones registradas por el Director para el año lectivo activo.

Elaborado por: Equipo BCEL.

Tabla 43 — Requisitos funcionales: Consulta del horario del curso

Campo	Contenido
ID	RF-D14
Nombre	Consulta del horario del curso
Descripción	El docente debe poder consultar el horario anual del curso que tiene asignado, mostrando todas las asignaturas y docentes del curso.
Entradas	Grado y paralelo del curso seleccionado.
Salidas	Horario completo del curso en formato de cuadrícula semanal.
Actor	Docente.
Prioridad	Media.
Criterio de verificación	El horario del curso muestra correctamente todos los períodos del día, las asignaturas asignadas y el nombre del docente correspondiente a cada período.

Elaborado por: Equipo BCEL.

Tabla 44 — Requisitos funcionales: Detalle del horario

Campo	Contenido
ID	RF-D15
Nombre	Detalle del horario
Descripción	El horario debe mostrar para cada entrada: período, hora de inicio y fin, asignatura y docente responsable por cada día de la semana.
Entradas	Horario configurado por el Director para el curso o docente consultado.
Salidas	Vista detallada del horario con todos los datos requeridos.
Actor	Docente, Director, Secretaría.
Prioridad	Media.
Criterio de verificación	Cada celda del horario muestra la hora, asignatura y docente correctamente sin solapamientos ni espacios sin información.

Elaborado por: Equipo BCEL.

3.4 Reportes del Microservicio Docente

Tabla 45 — Requisitos funcionales: Boleta trimestral (módulo docente)

Campo	Contenido
ID	RF-D16
Nombre	Generación de boleta trimestral
Descripción	El docente debe poder generar la boleta de calificaciones trimestral de cada estudiante de sus cursos asignados.
Entradas	Matrícula del estudiante y trimestre seleccionado.
Salidas	Boleta trimestral del estudiante en formato PDF.
Actor	Docente.
Prioridad	Alta.
Criterio de verificación	La boleta generada contiene las calificaciones de todas las asignaturas del estudiante en el trimestre, con el formato institucional establecido.

Elaborado por: Equipo BCEL.

Tabla 46 — Requisitos funcionales: Registro de Avance Estudiantil (RAE)

Campo	Contenido
ID	RF-D17
Nombre	Generación del Registro de Avance Estudiantil
Descripción	El docente tutor debe poder generar el RAE (Registro de Avance Estudiantil) de cada estudiante de su curso, incluyendo calificaciones, asistencia y observaciones.
Entradas	Matrícula del estudiante para el que se genera el RAE.
Salidas	Documento RAE generado en formato PDF con los datos académicos del estudiante.
Actor	Docente (tutor del curso).
Prioridad	Alta.
Criterio de verificación	El RAE generado contiene la información completa del período académico y cumple con el formato exigido por el Ministerio de Educación.

Elaborado por: Equipo BCEL.

Tabla 47 — Requisitos funcionales: Concentrado de notas

Campo	Contenido
ID	RF-D18
Nombre	Generación del concentrado de notas
Descripción	El docente debe poder generar el concentrado de notas por trimestre de su asignatura, mostrando las calificaciones de todos los estudiantes del curso.
Entradas	Asignación del docente (asignatura + grado) y trimestre seleccionado.
Salidas	Concentrado de notas en formato PDF con el listado completo de estudiantes y sus calificaciones.
Actor	Docente.
Prioridad	Alta.
Criterio de verificación	El concentrado de notas refleja correctamente las calificaciones de todos los estudiantes matriculados en el curso para la asignatura y trimestre seleccionados.

Elaborado por: Equipo BCEL.

4. Requerimientos No Funcionales

Los requerimientos no funcionales definen los atributos de calidad del sistema [7].

4.1 Seguridad

Tabla 48 — Requisito no funcional: Autenticación

Campo	Contenido
ID	RNF-01
Nombre	Autenticación de usuarios
Descripción	El sistema debe autenticar a los usuarios mediante usuario y contraseña antes de conceder acceso a cualquier funcionalidad.
Métrica / Restricción	Todos los accesos deben requerir autenticación previa; no existe acceso anónimo a recursos protegidos.
Actor	Todos los usuarios.
Prioridad	Alta.
Criterio de verificación	Un usuario no autenticado recibe el código HTTP 401 al intentar acceder a cualquier endpoint protegido.

Elaborado por: Equipo BCEL.

Tabla 49 — Requisito no funcional: Cifrado de contraseñas

Campo	Contenido
ID	RNF-02
Nombre	Cifrado de contraseñas con BCrypt
Descripción	Las contraseñas de los usuarios deben almacenarse cifradas utilizando el algoritmo BCrypt [8], nunca en texto plano.
Métrica / Restricción	Factor de costo BCrypt mínimo de 12.
Actor	Sistema.
Prioridad	Alta.
Criterio de verificación	Las contraseñas en la base de datos no son legibles; una inspección directa de la base de datos no revela las contraseñas originales.

Elaborado por: Equipo BCEL.

Tabla 50 — Requisito no funcional: Control de acceso basado en roles

Campo	Contenido
ID	RNF-03
Nombre	Control de acceso basado en roles (RBAC)
Descripción	El sistema debe controlar el acceso a los recursos mediante roles (RBAC), garantizando que cada usuario acceda únicamente a las funcionalidades que le corresponden [9].
Métrica / Restricción	Cobertura del 100 % de los endpoints con verificación de rol.
Actor	Sistema.
Prioridad	Alta.
Criterio de verificación	Un usuario con rol Docente no puede acceder a los módulos administrativos; el intento retorna el código HTTP 403.

Elaborado por: Equipo BCEL.

Tabla 51 — Requisito no funcional: Comunicación entre microservicios con JWT

Campo	Contenido
ID	RNF-05
Nombre	Autenticación inter-servicios con JWT
Descripción	La comunicación entre el Sistema Principal y el Microservicio Docente debe autenticarse mediante tokens JWT [10], garantizando que solo servicios autorizados puedan intercambiar datos.
Métrica / Restricción	Todos los requests entre microservicios deben incluir un JWT válido en el header de autorización.
Actor	Sistema.
Prioridad	Alta.
Criterio de verificación	Un request sin token JWT válido entre microservicios es rechazado con código HTTP 401.

Elaborado por: Equipo BCEL.

Tabla 52 — Requisito no funcional: TLS 1.3

Campo	Contenido
ID	RNF-17
Nombre	Comunicación cifrada con TLS 1.3
Descripción	Toda la comunicación entre el cliente y el servidor, y entre los microservicios, debe cifrarse utilizando el protocolo TLS versión 1.3 [11].
Métrica / Restricción	Versión mínima de TLS permitida: 1.3. Las versiones anteriores deben ser rechazadas.
Actor	Sistema.
Prioridad	Alta.
Criterio de verificación	Las herramientas de análisis de tráfico confirman que toda la comunicación utiliza TLS 1.3; los intentos de conexión con versiones anteriores son rechazados.

Elaborado por: Equipo BCEL.

Tabla 53 — Requisito no funcional: Rate limiting

Campo	Contenido
ID	RNF-18
Nombre	Rate limiting en el API Gateway
Descripción	El API Gateway debe implementar rate limiting para limitar el número de requests por cliente en un intervalo de tiempo, previniendo ataques de denegación de servicio (DDoS).
Métrica / Restricción	Máximo 100 requests por minuto por dirección IP.
Actor	Sistema.
Prioridad	Alta.
Criterio de verificación	Al superar el límite, el sistema retorna el código HTTP 429 (Too Many Requests) sin procesar la solicitud.

Elaborado por: Equipo BCEL.

Tabla 54 — Requisito no funcional: Logs de auditoría

Campo	Contenido
ID	RNF-19
Nombre	Logs de auditoría con integridad HMAC
Descripción	El sistema debe registrar logs de auditoría de todas las operaciones críticas (autenticación, modificación de datos y generación de reportes), firmados con HMAC para garantizar su integridad.
Métrica / Restricción	100 % de las operaciones críticas registradas con marca de tiempo, usuario, acción y hash HMAC.
Actor	Sistema.
Prioridad	Alta.
Criterio de verificación	Cualquier alteración de un registro de auditoría es detectada mediante la verificación del hash HMAC.

Elaborado por: Equipo BCEL.

4.2 Disponibilidad

Tabla 55 — Requisito no funcional: Disponibilidad en horario escolar

Campo	Contenido
ID	RNF-06
Nombre	Disponibilidad en horario escolar
Descripción	El sistema debe estar disponible durante el horario escolar de la institución (07:00–17:00, días hábiles), garantizando continuidad en las operaciones académicas.
Métrica / Restricción	Disponibilidad mínima del 99 % en el horario definido.
Actor	Todos los usuarios.
Prioridad	Alta.
Criterio de verificación	El sistema no presenta interrupciones no planificadas durante el horario escolar; las caídas son menores a 5 minutos por día.

Elaborado por: Equipo BCEL.

Tabla 56 — Requisito no funcional: Operación con conectividad limitada

Campo	Contenido
ID	RNF-07
Nombre	Operación con conectividad limitada
Descripción	El sistema debe ser funcional en condiciones de conectividad limitada, dado que la institución se ubica en zona rural con servicio de fibra óptica CNT sujeto a cortes eventuales.
Métrica / Restricción	Tiempo máximo de carga de páginas: 5 segundos en condiciones de ancho de banda reducido (1 Mbps).
Actor	Todos los usuarios.
Prioridad	Alta.
Criterio de verificación	El sistema opera correctamente con un ancho de banda de 1 Mbps; las páginas cargan dentro del tiempo máximo establecido.

Elaborado por: Equipo BCEL.

4.3 Usabilidad

Tabla 57 — Requisito no funcional: Interfaz intuitiva

Campo	Contenido
ID	RNF-08
Nombre	Interfaz intuitiva sin capacitación extensa
Descripción	La interfaz del sistema debe ser lo suficientemente intuitiva para que los usuarios puedan realizar sus tareas sin necesidad de capacitación técnica extensa.
Métrica / Restricción	El 80 % de los usuarios debe completar tareas principales sin asistencia técnica en la primera sesión.
Actor	Todos los usuarios.
Prioridad	Alta.
Criterio de verificación	Las pruebas de usabilidad con usuarios reales de la institución muestran que el 80 % completa las tareas clave sin apoyo externo.

Elaborado por: Equipo BCEL.

Tabla 58 — Requisito no funcional: Diseño responsivo

Campo	Contenido
ID	RNF-09
Nombre	Diseño responsivo para dispositivos móviles
Descripción	La interfaz del sistema debe adaptarse a diferentes tamaños de pantalla, siendo completamente funcional en teléfonos móviles y tabletas.
Métrica / Restricción	Resoluciones mínimas soportadas: 360×640 px (móvil) y 768×1024 px (tableta).
Actor	Todos los usuarios.
Prioridad	Media.
Criterio de verificación	Las funcionalidades principales son accesibles y operables en las resoluciones mínimas definidas sin pérdida de funcionalidad.

Elaborado por: Equipo BCEL.

Tabla 59 — Requisito no funcional: Idioma español

Campo	Contenido
ID	RNF-10
Nombre	Sistema completamente en español
Descripción	Toda la interfaz del sistema, incluyendo mensajes de error, etiquetas y documentación de ayuda, debe estar en idioma español.
Métrica / Restricción	Cobertura del 100 % de los textos de la interfaz en español.
Actor	Todos los usuarios.
Prioridad	Alta.
Criterio de verificación	No existe ningún texto, mensaje o etiqueta en idioma diferente al español en ninguna pantalla del sistema.

Elaborado por: Equipo BCEL.

4.4 Rendimiento

Tabla 60 — Requisito no funcional: Tiempo de respuesta

Campo	Contenido
ID	RNF-11
Nombre	Tiempo de respuesta máximo de 3 segundos
Descripción	El sistema debe responder a cualquier operación del usuario en un tiempo máximo de 3 segundos bajo condiciones normales de uso.
Métrica / Restricción	Tiempo de respuesta ≤ 3 segundos para el percentil 95 de las solicitudes.
Actor	Todos los usuarios.
Prioridad	Alta.
Criterio de verificación	Las pruebas de carga con 20 usuarios simultáneos confirman que el 95 % de las solicitudes responden en 3 segundos o menos.

Elaborado por: Equipo BCEL.

Tabla 61 — Requisito no funcional: Concurrencia

Campo	Contenido
ID	RNF-12
Nombre	Soporte de acceso simultáneo de 20 usuarios
Descripción	El sistema debe soportar el acceso simultáneo de al menos 20 usuarios sin degradación del rendimiento.
Métrica / Restricción	20 usuarios concurrentes con tiempo de respuesta ≤ 3 segundos.
Actor	Todos los usuarios.
Prioridad	Alta.
Criterio de verificación	Las pruebas de carga con 20 usuarios simultáneos no producen errores ni tiempos de respuesta superiores al umbral establecido.

Elaborado por: Equipo BCEL.

4.5 Mantenibilidad

Tabla 62 — Requisito no funcional: Contenerización con Docker

Campo	Contenido
ID	RNF-13
Nombre	Contenerización con Docker
Descripción	Cada componente del sistema debe estar contenerizado utilizando Docker, facilitando el despliegue reproducible en cualquier entorno [12].
Métrica / Restricción	Todos los servicios deben ejecutarse como contenedores Docker orquestados con Docker Compose.
Actor	Equipo de Soporte Técnico.
Prioridad	Alta.
Criterio de verificación	El sistema puede desplegarse completamente ejecutando <code>docker-compose up</code> desde cualquier servidor con Docker instalado.

Elaborado por: Equipo BCEL.

Tabla 63 — Requisito no funcional: Despliegue independiente

Campo	Contenido
ID	RNF-14
Nombre	Despliegue independiente de microservicios
Descripción	Cada microservicio debe poder desplegarse, actualizarse o reiniciarse de forma independiente sin afectar el funcionamiento de los demás servicios.
Métrica / Restricción	La actualización o caída del Microservicio Docente no interrumpe el Sistema Principal.
Actor	Equipo de Soporte Técnico.
Prioridad	Alta.
Criterio de verificación	Al detener el Microservicio Docente, el Sistema Principal continúa respondiendo correctamente a sus propias solicitudes.

Elaborado por: Equipo BCEL.

4.6 Escalabilidad

Tabla 64 — Requisito no funcional: Extensibilidad de la arquitectura

Campo	Contenido
ID	RNF-16
Nombre	Extensibilidad para nuevos microservicios
Descripción	La arquitectura del sistema debe permitir la incorporación de nuevos microservicios (p. ej., fichas psicológicas, biblioteca, pagos) sin modificar ni interrumpir los servicios existentes [2].
Métrica / Restricción	Un nuevo microservicio puede integrarse al API Gateway sin modificar el código del Sistema Principal ni del Microservicio Docente.
Actor	Equipo de Desarrollo.
Prioridad	Media.
Criterio de verificación	La incorporación de un servicio de demostración al API Gateway no produce errores ni cambios de comportamiento en los servicios existentes.

Elaborado por: Equipo BCEL.

5. Contrato API REST

El diseño del contrato API REST sigue los principios arquitecturales REST definidos por Fielding [13] y las convenciones de nomenclatura de recursos REST [13].

5.1 URL Base

Sistema Principal : <https://api.sga.provinciasunidas.edu.ec/api/v1>

Microservicio Doc. : <https://api.sga.provinciasunidas.edu.ec/docente/v1>

5.2 Sistema Principal (Puerto 8080)

5.2.1. Autenticación

Método	Endpoint	Descripción	Roles
POST	/auth/login	Iniciar sesión	Público
POST	/auth/logout	Cerrar sesión	Autenticado
POST	/auth/refresh	Renovar token JWT	Autenticado
POST	/auth/cambiar-password	Cambiar contraseña	Autenticado

Cuadro 1: Endpoints de Autenticación

5.2.2. Usuarios

Método	Endpoint	Descripción	Roles
GET	/usuarios	Listar todos	Dir., Soporte
GET	/usuarios/{id}	Ver por ID	Dir., Soporte
POST	/usuarios	Crear usuario	Dir., Soporte
PUT	/usuarios/{id}	Editar usuario	Dir., Soporte
PATCH	/usuarios/{id}/estado	Activar/desactivar	Dir., Soporte
POST	/usuarios/{id}/reset-password	Resetear contraseña	Dir., Soporte

Cuadro 2: Endpoints de Usuarios

5.2.3. Años Lectivos

Método	Endpoint	Descripción	Roles
GET	/anos-lectivos	Listar todos	Todos
GET	/anos-lectivos/{id}	Ver por ID	Todos
POST	/anos-lectivos	Crear año	Director
PUT	/anos-lectivos/{id}	Editar año	Director
DELETE	/anos-lectivos/{id}	Eliminar	Director
PATCH	/anos-lectivos/{id}/establecer-actual	Marcar como actual	Director

Cuadro 3: Endpoints de Años Lectivos

5.2.4. Estudiantes

Método	Endpoint	Descripción	Roles
GET	/estudiantes	Listar todos	Dir., Sec.
GET	/estudiantes/{id}	Ver por ID	Dir., Sec.
GET	/estudiantes/buscar?q=	Buscar por nombre/cédula	Dir., Sec.
POST	/estudiantes	Registrar	Dir., Sec.
PUT	/estudiantes/{id}	Editar	Dir., Sec.
GET	/estudiantes/{id}/representante	Ver representante	Dir., Sec.
PUT	/estudiantes/{id}/representante	Actualizar representante	Dir., Sec.

Cuadro 4: Endpoints de Estudiantes

5.2.5. Matrículas

Método	Endpoint	Descripción	Roles
GET	/matriculas	Listar todas	Dir., Sec.
GET	/matriculas/{id}	Ver por ID	Dir., Sec.
GET	/matriculas/ano-lectivo/{idAno}	Por año lectivo	Dir., Sec.
GET	/matriculas/grado/{idGrado}	Por grado	Dir., Sec., Doc.
POST	/matriculas	Crear	Dir., Sec.
PATCH	/matriculas/{id}/estado	Cambiar estado	Dir., Sec.
DELETE	/matriculas/{id}	Eliminar	Director

Cuadro 5: Endpoints de Matrículas

5.2.6. Grados y Asignaturas

Método	Endpoint	Descripción	Roles
Grados			
GET	/grados	Listar todos	Todos
GET	/grados/{id}	Ver por ID	Todos
POST	/grados	Crear grado	Director
PUT	/grados/{id}	Editar grado	Director
PATCH	/grados/{id}/estado	Activar/desactivar	Director
Asignaturas			
GET	/asignaturas	Listar todas	Todos
GET	/asignaturas/{id}	Ver por ID	Todos

GET	/asignaturas/nivel/{nivel}	Por nivel educativo	Todos
POST	/asignaturas	Crear asignatura	Director
PUT	/asignaturas/{id}	Editar asignatura	Director

Cuadro 6: Endpoints de Grados y Asignaturas

5.2.7. Asignaciones y Horarios

Método	Endpoint	Descripción	Roles
Asignaciones (Carga Horaria)			
GET	/asignaciones	Listar todas	Director
GET	/asignaciones/docente/{idDocente}	Por docente	Dir., Doc.
GET	/asignaciones/grado/{idGrado}	Por grado	Dir., Sec.
GET	/asignaciones/ano-lectivo/{idAno}	Por año lectivo	Director
POST	/asignaciones	Crear	Director
PUT	/asignaciones/{id}	Editar	Director
DELETE	/asignaciones/{id}	Eliminar	Director
Horarios			
GET	/horarios/grado/{idGrado}	Horario del curso	Todos
GET	/horarios/docente/{idDocente}	Horario del docente	Dir., Doc.
POST	/horarios	Crear entrada	Director
PUT	/horarios/{id}	Editar entrada	Director
DELETE	/horarios/{id}	Eliminar entrada	Director

Cuadro 7: Endpoints de Asignaciones y Horarios

5.2.8. Reportes — Sistema Principal

Método	Endpoint	Descripción	Roles
GET	/reportes/boleta/{idMatricula}/{trimestre}	Boleta por estudiante	Dir., Sec.
GET	/reportes/acta/{idGrado}/{trimestre}	Acta por grado	Dir., Sec.
GET	/reportes/asistencia/{idGrado}	Asistencia general	Dir., Sec.
GET	/reportes/matriculas/{idAno}	Matrículas por año	Dir., Sec.

Cuadro 8: Endpoints de Reportes — Sistema Principal

5.3 Microservicio Docente (Puerto 8081)

5.3.1. Actividades y Calificaciones

Método	Endpoint	Descripción	Roles
Actividades			
GET	/actividades/asignacion/{idAsignacion}	Por asignación	Doc., Dir.
GET	/actividades/{id}	Por ID	Doc., Dir.
POST	/actividades	Crear actividad	Docente
PUT	/actividades/{id}	Editar actividad	Docente
DELETE	/actividades/{id}	Eliminar actividad	Docente
Calificaciones			
GET	/calificaciones/actividad/{idActividad}	Por actividad	Doc., Dir.
GET	/calificaciones/matricula/{idMatricula}	Por estudiante	Doc., Dir., Sec.
GET	/calificaciones/trimestre/{idAsignacion}/trimestre	Por trimestre	Doc., Dir.
POST	/calificaciones	Ingresar	Docente
PUT	/calificaciones/{id}	Editar	Docente
GET	/calificaciones/promedio/{idMatricula}/calificaciones/{trimestre}	Por estudiante y trimestre promedio	Doc., Dir.

Cuadro 9: Endpoints de Actividades y Calificaciones

5.3.2. Asistencia y Periodos de Evaluación

Método	Endpoint	Descripción	Roles
Asistencia			
GET	/asistencias/asignacion/{id}/fecha/{fecha}	Por fecha y curso	Doc., Dir.
GET	/asistencias/matricula/{idMatricula}	Historial estudiante	Doc., Dir., Sec.
GET	/asistencias/trimestre/{idAsignacion}/trimestre	Resumen trimestral	Doc., Dir.
POST	/asistencias	Registrar	Docente
PUT	/asistencias/{id}	Editar registro	Docente

Períodos de Evaluación			
GET	/periodos/ano-lectivo/{idAño}	Listar períodos	Todos
GET	/periodos/{id}	Ver por ID	Todos
POST	/periodos	Crear período	Director
PUT	/periodos/{id}	Editar período	Director
PATCH	/periodos/{id}/estado	Activar/cerrar	Director

Cuadro 10: Endpoints de Asistencia y Períodos de Evaluación

5.3.3. Reportes — Microservicio Docente

Método	Endpoint	Descripción	Roles
GET	/reportes/boleta/{idMatricula}/{trimestre}	Boleta trimestral	Doc., Dir., Sec.
GET	/reportes/rae/{idMatricula}	RAE del estudiante	Doc., Dir.
GET	/reportes/concentrado/{idAsignacion}/{trimestre}	Concentrado de notas	Doc., Dir.
GET	/reportes/asistencia/{idAsignacion}/{trimestre}	Reporte de asistencia	Doc., Dir.

Cuadro 11: Endpoints de Reportes — Microservicio Docente

5.4 Códigos de Respuesta HTTP

Código	Descripción
200	OK — solicitud exitosa
201	Created — recurso creado exitosamente
204	No Content — recurso eliminado correctamente
400	Bad Request — datos inválidos o malformados
401	Unauthorized — sin token o token inválido
403	Forbidden — sin permisos para el recurso solicitado
404	Not Found — recurso no encontrado
409	Conflict — duplicado (ej.: matrícula ya existe en ese año lectivo)
429	Too Many Requests — límite de rate limiting superado
500	Internal Server Error — error no controlado del servidor

Cuadro 12: Códigos de respuesta HTTP utilizados en el API

6. Arquitectura del Sistema

La arquitectura de microservicios divide el sistema en dos servicios independientes, comunicados a través de un API Gateway, siguiendo los patrones descritos por Richardson [4] y Newman [2].

6.1 Servicios

Servicio	Puerto	Responsabilidad
Sistema Principal	8080	Gestión administrativa: usuarios, matrículas, grados, asignaturas y horarios.
Microservicio Docente	8081	Gestión pedagógica: calificaciones, asistencia y reportes docentes.
PostgreSQL	—	Base de datos con dos schemas separados: sga_main y sga_docente [14].
Docker Compose	—	Orquestación de contenedores; cada servicio se despliega en un contenedor independiente [12, 15].

Cuadro 13: Servicios del sistema

6.2 Justificación de la Arquitectura

1. **Separación de dominios:** la gestión administrativa y la pedagógica presentan ciclos de cambio y requisitos de disponibilidad distintos [3].
2. **Resiliencia:** un fallo en el microservicio docente no interrumpe las operaciones administrativas [2].
3. **Extensibilidad:** futuros módulos pueden incorporarse sin modificar los servicios existentes [4].

7. Marco Normativo

El sistema se diseña en conformidad con:

- El currículo nacional y los lineamientos de evaluación del Ministerio de Educación del Ecuador [1].
- La Ley Orgánica de Educación Intercultural (LOEI) [1].
- El Reglamento General a la LOEI [16].
- La Ley Orgánica de Protección de Datos Personales del Ecuador [17].

8. Registro de Cambios — Entrega 2

Versión	Sección modificada	Descripción del cambio	Motivo
2.0	Portada	Actualización a versión 2.0 — Entrega 2	Nueva entrega académica
2.0	§8 (nuevo)	Registro de cambios	Trazabilidad del documento
2.0	§9 (nuevo)	Diagramas C4 Niveles 1, 2 y 3	Retroalimentación docente: faltaba representación visual de la arquitectura
2.0	§10 (nuevo)	Architecture Decision Records (ADR)	Retroalimentación docente: justificar formalmente las decisiones de diseño
2.0	§11 (nuevo)	Modelo de datos distribuido (ER)	Formalizar la estructura de ambos schemas PostgreSQL
2.0	§12 (nuevo)	Estrategia de contenerización Docker	Detallar Dockerfiles y docker-compose.yml completos
2.0	§13 (nuevo)	Protocolo experimental reproducible	Documentar procedimiento de despliegue en ≤ 30 minutos
2.0	Bibliografía	Adición de 4 referencias nuevas (ADR, C4, Docker Compose, ER)	Soporte a secciones añadidas

Cuadro 14: Registro de cambios entre Entrega 1 y Entrega 2

9. Diagramas C4 de Arquitectura

El modelo C4 [18] proporciona cuatro niveles de abstracción para documentar la arquitectura de software: Contexto (Nivel 1), Contenedores (Nivel 2), Componentes (Nivel 3) y Código (Nivel 4). En este documento se presentan los tres primeros niveles.

9.1 C4 Nivel 1 — Contexto del Sistema

El diagrama de contexto muestra el SGA en relación con sus usuarios y sistemas externos, sin entrar en detalles de implementación.

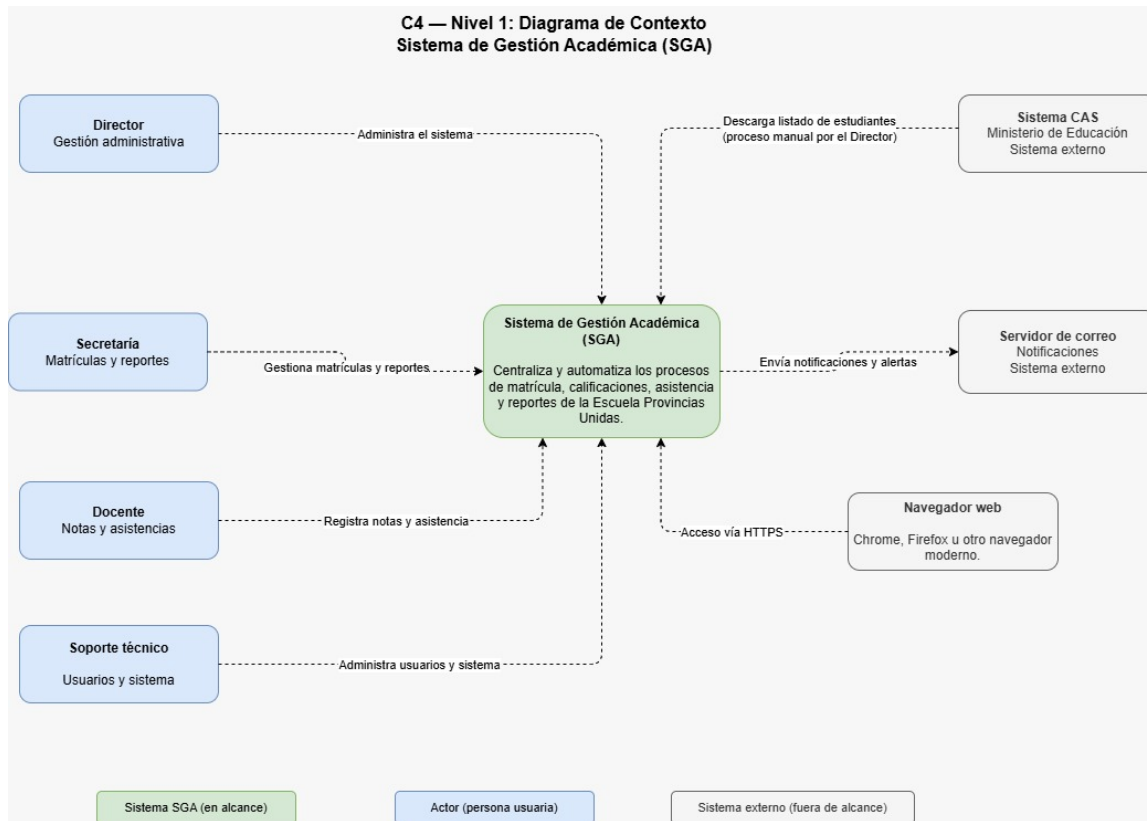


Figura 1: C4 Nivel 1 — Diagrama de Contexto del SGA

Descripción de elementos:

Elemento	Tipo	Descripción
SGA	Sistema	Plataforma central de gestión académica desarrollada para la Escuela «Provincias Unidas».
Director	Actor	Administra usuarios, grados, horarios y genera reportes institucionales.
Secretaría	Actor	Gestiona matrículas, consulta información de estudiantes y genera reportes.
Docente	Actor	Registra calificaciones y asistencia de sus cursos asignados.
Soporte Técnico	Actor	Mantiene el sistema, gestiona usuarios y monitorea logs.
Sistema CAS	Externo	Sistema del Ministerio de Educación; fuente del listado oficial de estudiantes matriculados.
Servidor de correo	Externo	Recibe notificaciones automáticas del SGA (alertas, contraseñas provisionales).
Navegador web	Externo	Cliente utilizado por todos los actores para acceder a la interfaz del SGA.

Cuadro 15: Elementos del diagrama de Contexto C4 — Nivel 1

9.2 C4 Nivel 2 — Contenedores

El diagrama de contenedores descompone el SGA en sus procesos y almacenes de datos ejecutables, mostrando las tecnologías y protocolos de comunicación.

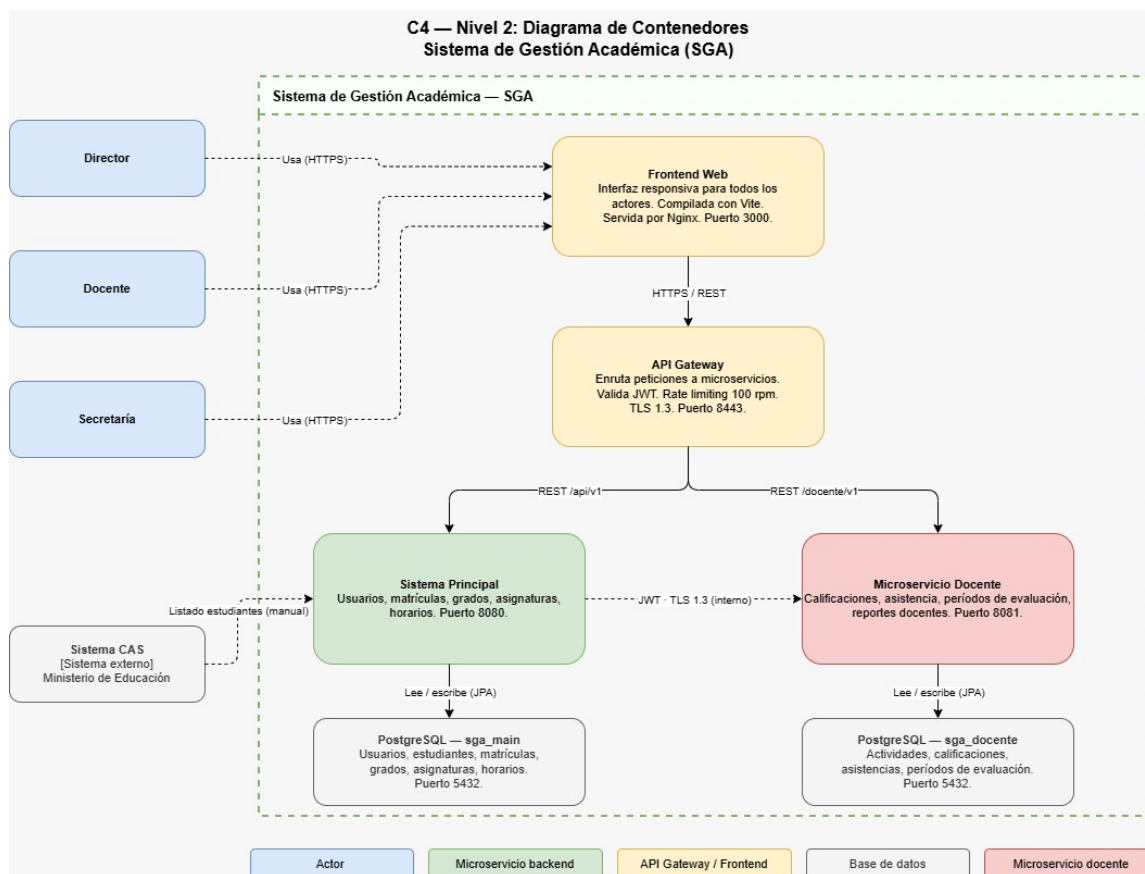


Figura 2: C4 Nivel 2 — Diagrama de Contenedores del SGA

Contenedor	Puerto	Tecnología	Responsabilidad
Frontend web	3000	React · SPA	Interfaz responsiva para todos los actores.
API Gateway	443	Spring Cloud Gateway	Enrutamiento, validación JWT y rate limiting.
Sistema Principal	8080	Spring Boot	Gestión administrativa: usuarios, matrículas, grados, horarios.
Microservicio Docente	8081	Spring Boot	Gestión pedagógica: calificaciones y asistencia.
PostgreSQL sga_main	5432	PostgreSQL 16	Persistencia del dominio administrativo.
PostgreSQL sga_docente	5432	PostgreSQL 16	Persistencia del dominio pedagógico.

Cuadro 16: Contenedores del SGA — Nivel 2

9.3 C4 Nivel 3 — Componentes del Sistema Principal

El diagrama de componentes descompone el Sistema Principal (puerto 8080) en sus capas internas, siguiendo el patrón de arquitectura en capas (Controller → Service → Repository).

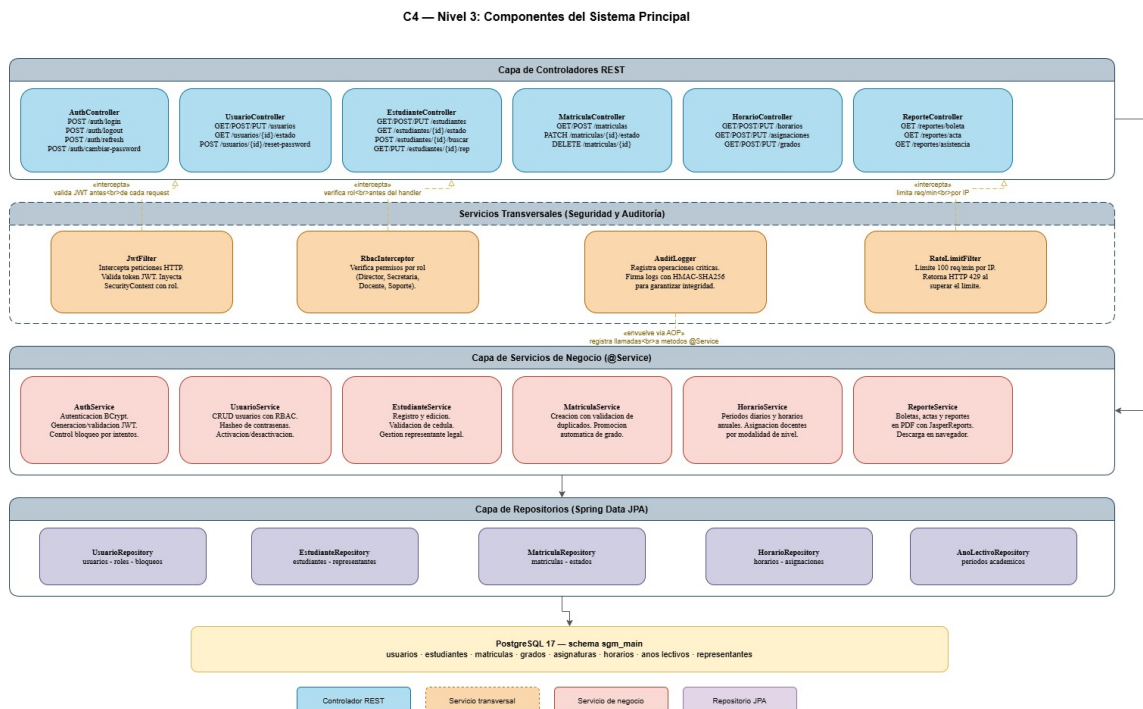


Figura 3: C4 Nivel 3 — Componentes del Sistema Principal (Puerto 8080)

9.4 C4 Nivel 3 — Componentes del Microservicio Docente

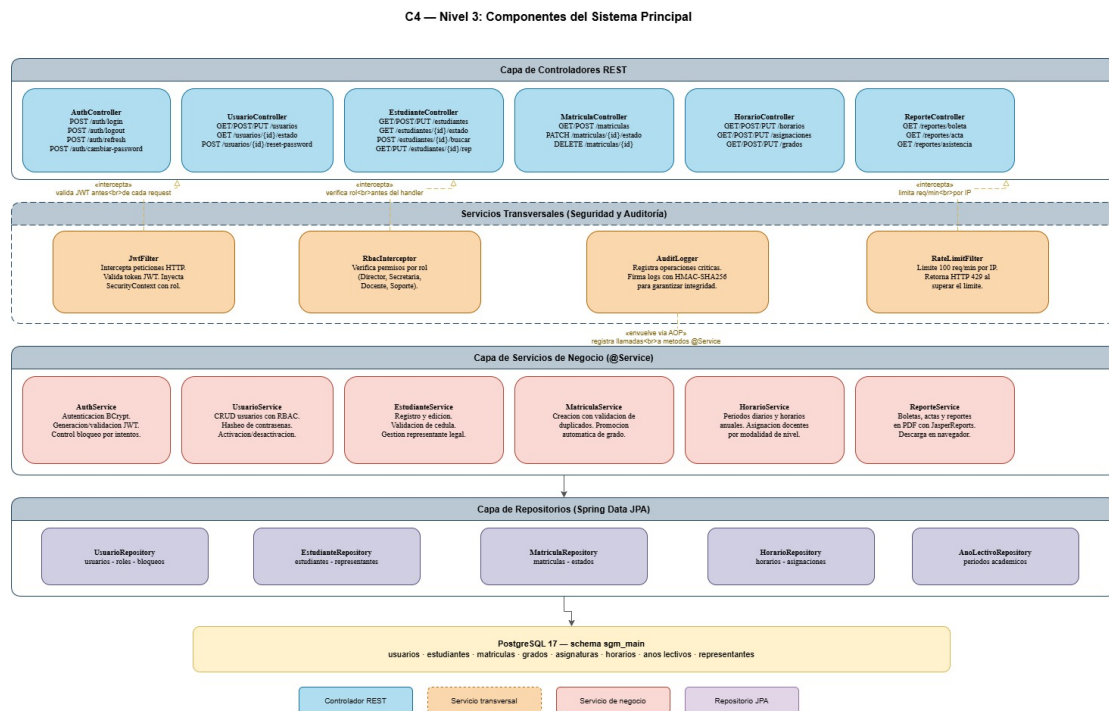


Figura 4: C4 Nivel 3 — Componentes del Microservicio Docente (Puerto 8081)

10. Architecture Decision Records (ADR)

Los siguientes registros de decisiones arquitectónicas documentan las elecciones de diseño más relevantes del Sistema de Gestión Académica, siguiendo el formato propuesto por Nygard [19]: contexto, opciones consideradas, decisión adoptada y consecuencias esperadas.

10.1 Arquitectura general

ADR-001 — Adopción de arquitectura de microservicios

Estado: Aceptado

Contexto: La Escuela *Provincias Unidas* presenta dos dominios de negocio claramente diferenciados: la gestión administrativa (usuarios, matrículas, horarios) y la gestión pedagógica (calificaciones, asistencia). Ambos dominios tienen ciclos de cambio, equipos responsables y requisitos de disponibilidad distintos [?].

Decisión: Se adopta una arquitectura de microservicios con dos servicios independientes: el Sistema Principal (puerto 8080) y el Microservicio Docente (puerto 8081). Esta separación permite despliegues independientes, escalabilidad diferenciada y que un fallo en el módulo docente no interrumpa las operaciones administrativas [?].

Consecuencias: Mayor complejidad operativa (red, despliegue, trazabilidad distribuida) compensada por resiliencia y extensibilidad a largo plazo [4].

ADR-002 — API Gateway como punto de entrada único

Estado: Aceptado

Contexto: Con dos microservicios expuestos en puertos distintos, los clientes (navegador, aplicaciones futuras) necesitarían conocer la dirección de cada servicio individualmente, lo que genera acoplamiento de red y dificulta aplicar políticas transversales (autenticación, *rate limiting*, TLS) [4].

Decisión: Se introduce un API Gateway (Nginx o Spring Cloud Gateway) como único punto de entrada. El Gateway enruta `/api/v1` al Sistema Principal y `/docente/v1` al Microservicio Docente, aplica TLS 1.3 en el perímetro y valida el token JWT antes de reenviar la petición [13].

Consecuencias: Punto único de fallo potencial; se mitiga con reintentos automáticos y healthchecks en Docker Compose.

ADR-003 — REST sobre gRPC para comunicación cliente-servidor

Estado: Aceptado

Contexto: El sistema requiere interoperabilidad con navegadores web sin necesidad de bibliotecas cliente especializadas. Se evaluaron REST [13], gRPC [?] y GraphQL [?].

Decisión: Se adopta REST con JSON para la comunicación cliente-servidor. REST es el estándar de facto para APIs web, cuenta con soporte nativo en navegadores y herramientas de depuración ampliamente disponibles. gRPC fue descartado por la complejidad de integración en navegadores (*grpc-web*); GraphQL, por la curva de aprendizaje y el alcance acotado del proyecto [?].

Consecuencias: Mayor verbosidad de mensajes frente a gRPC; aceptable dado el volumen de usuarios concurrentes previsto (≤ 20).

ADR-004 — Comunicación síncrona inter-microservicios

Estado: Aceptado

Contexto: El Microservicio Docente requiere consultar datos de matrículas y asignaciones almacenados en el Sistema Principal. Se evaluaron llamadas REST síncronas, mensajería asíncrona (RabbitMQ) y compartir base de datos [14].

Decisión: Se opta por llamadas REST síncronas entre microservicios, autenticadas con JWT y cifradas con TLS 1.3 [?]. La mensajería asíncrona fue descartada por agregar infraestructura (broker) innecesaria para el volumen de la institución. Compartir base de datos fue descartado por violar el principio de independencia de datos [?].

Consecuencias: Dependencia temporal en tiempo de respuesta; aceptable dado que ambos servicios corren en la misma red Docker interna.

10.2 Base de datos

ADR-005 — PostgreSQL como motor de base de datos

Estado: Aceptado

Contexto: El sistema gestiona datos relacionales con integridad referencial estricta (estudiantes, matrículas, calificaciones). Se evaluaron PostgreSQL, MySQL y MongoDB [14].

Decisión: Se selecciona PostgreSQL 16 por su soporte robusto de transacciones ACID, tipos de datos avanzados (UUID, JSONB), extensibilidad y licencia de código abierto. MySQL fue descartado por limitaciones históricas en conformidad SQL; MongoDB, por ser inadecuado para datos fuertemente relacionales [14].

Consecuencias: Requiere mayor configuración inicial frente a MySQL; compensado por confiabilidad y soporte comunitario extenso.

ADR-006 — Dos schemas separados en una instancia PostgreSQL

Estado: Aceptado

Contexto: Se debatió entre: (a) una sola base de datos compartida, (b) dos bases de datos independientes en instancias separadas, y (c) dos schemas en la misma instancia [14].

Decisión: Se adoptan dos schemas separados (`sga_main` y `sga_docente`) en una única instancia PostgreSQL. Esta opción preserva la independencia lógica de datos, simplifica la operación (un solo contenedor de base de datos) y reduce el consumo de recursos en el entorno de la institución [?].

Consecuencias: Un fallo de la instancia afecta a ambos schemas; mitigado con respaldos automáticos diarios en Docker volume.

ADR-007 — UUID como clave primaria en todas las entidades

Estado: Aceptado

Contexto: El uso de enteros autoincrementales expone la cantidad de registros y facilita ataques de enumeración. Se evaluaron enteros secuenciales, `SERIAL` y `UUID v4` [?].

Decisión: Se adopta `UUID v4` generado en la capa de aplicación (Spring) como clave primaria. Esto garantiza unicidad global, facilita la futura federación de datos y elimina la dependencia de secuencias de base de datos [14].

Consecuencias: Mayor tamaño de índice frente a enteros; irrelevante para el volumen de datos esperado (< 500 estudiantes).

ADR-008 — Spring Data JPA como capa de acceso a datos

Estado: Aceptado

Contexto: Se evaluaron JDBC puro, MyBatis y Spring Data JPA para la capa de persistencia. La productividad del equipo y el tipo de consultas (CRUD estándar con pocas consultas complejas) influyeron en la decisión [?].

Decisión: Se adopta Spring Data JPA con Hibernate como proveedor. JPA abstraer el SQL estándar, reduce el código repetitivo y facilita la validación de entidades con Bean Validation. Para consultas complejas (reportes) se usarán `@Query` con JPQL o SQL nativo [?].

Consecuencias: Curva de aprendizaje de mapeo objeto-relacional; compensado por la amplia documentación y soporte de la comunidad Spring.

10.3 Seguridad

ADR-009 — JWT para autenticación inter-microservicios y cliente-servidor

Estado: Aceptado

Contexto: El sistema necesita autenticar peticiones tanto desde el navegador (cliente externo) como entre microservicios (comunicación interna). Se evaluaron sesiones con cookies, JWT y OAuth 2.0 [?].

Decisión: Se adopta JWT (RFC 7519) firmado con HMAC-SHA256. El token contiene el identificador de usuario, su rol y la fecha de expiración. El API Gateway valida la firma antes de reenviar la petición; los microservicios confían en el token validado por el Gateway, evitando doble validación [?].

Consecuencias: Los tokens no pueden revocarse antes de su expiración sin una lista negra; se mitiga con tiempos de expiración cortos (30 min) y tokens de refresco (7 días).

ADR-010 — BCrypt para almacenamiento de contraseñas

Estado: Aceptado

Contexto: El almacenamiento de contraseñas en texto plano o con hashes no salados (MD5, SHA-1) es vulnerable a ataques de diccionario y tablas arcoíris [?].

Decisión: Se adopta BCrypt con factor de coste 12 para el hash de contraseñas, siguiendo la recomendación de Provos y Mazières [?]. BCrypt incorpora sal aleatoria por diseño y su coste computacional es ajustable para resistir ataques de fuerza bruta a medida que el hardware mejora.

Consecuencias: Mayor tiempo de respuesta en login (≈ 200 ms); aceptable para la frecuencia de autenticación del sistema.

ADR-011 — Control de acceso basado en roles (RBAC)

Estado: Aceptado

Contexto: El sistema tiene cuatro tipos de usuarios con permisos distintos: Director, Secretaría, Docente y Soporte Técnico. Se evaluaron RBAC, ABAC (*Attribute-Based Access Control*) y listas de control de acceso (ACL) [?].

Decisión: Se implementa RBAC conforme al modelo de Ferraiolo et al. [?] mediante anotaciones `@PreAuthorize` de Spring Security. Los roles se almacenan en el token JWT y se verifican por el `RbacInterceptor` antes de invocar cada controlador. ABAC fue descartado por complejidad innecesaria dado el número reducido de roles.

Consecuencias: Cambios de permisos requieren actualizar el código de anotaciones; aceptable dado que los roles son estables.

ADR-012 — TLS 1.3 para toda la comunicación en red

Estado: Aceptado

Contexto: La institución se ubica en zona rural con fibra óptica CNT sujeta a cortes. El tráfico HTTP sin cifrar expone credenciales y datos de estudiantes menores de edad, violando la Ley Orgánica de Protección de Datos Personales del Ecuador [?].

Decisión: Se adopta TLS 1.3 (RFC 8446) [?] en el API Gateway para todo el tráfico externo. La comunicación interna entre microservicios (dentro de la red Docker) también usa TLS 1.3 con certificados autofirmados gestionados por Docker secrets.

Consecuencias: Requiere gestión de certificados; en producción se usará Let's Encrypt con renovación automática vía Certbot.

ADR-013 — Rate limiting en el API Gateway

Estado: Aceptado

Contexto: La institución opera con conectividad limitada. Un ataque de denegación de servicio (DDoS) o un cliente mal configurado podría saturar el sistema durante el horario escolar [13].

Decisión: Se implementa *rate limiting* en el API Gateway: máximo 100 peticiones por minuto por dirección IP, con respuesta 429 Too Many Requests. El umbral es configurable por entorno mediante variables de Docker Compose.

Consecuencias: Un usuario legítimo con IP dinámica compartida podría verse afectado; se mitiga con un umbral conservador y lista blanca para IPs internas.

ADR-014 — Registro de auditoría con HMAC para integridad

Estado: Aceptado

Contexto: Los datos académicos de menores son sensibles. La normativa ecuatoriana exige trazabilidad de operaciones sobre datos personales [?]. Se evaluaron logs planos, logs firmados y soluciones de auditoría externas.

Decisión: Se implementa un AuditLogger como aspecto AOP que firma cada entrada de log con HMAC-SHA256 usando una clave almacenada en Docker secret. Esto garantiza que los registros no puedan modificarse sin detección [?].

Consecuencias: Ligero incremento en latencia por operación de firma; irrelevante dado el volumen de operaciones (< 20 usuarios concurrentes).

10.4 Contenerización y despliegue

ADR-015 — Docker Compose sobre Kubernetes

Estado: Aceptado

Contexto: Se evaluaron Docker Compose [?], Kubernetes [?] y despliegue en máquina virtual sin contenedores para el entorno de producción de la institución.

Decisión: Se adopta Docker Compose v2 como orquestador. Kubernetes fue descartado por su complejidad operativa y los requisitos de hardware (mínimo 2 nodos con 2 vCPU y 4 GB RAM cada uno) incompatibles con la infraestructura disponible en la institución [?]. Docker Compose permite reproducir el entorno completo con un único comando.

Consecuencias: Sin escalado horizontal automático; aceptable para el volumen actual (≤ 20 usuarios concurrentes).

ADR-016 — Un Dockerfile por microservicio

Estado: Aceptado

Contexto: Empaquetar todos los servicios en una sola imagen simplifica el flujo CI/CD pero viola el principio de responsabilidad única y dificulta el despliegue independiente [?].

Decisión: Cada microservicio (Sistema Principal, Microservicio Docente, Frontend) tiene su propio `Dockerfile` con imagen base `eclipse-temurin:21-jre-alpine` para los servicios Java y `node:20-alpine` para el frontend. Las imágenes se construyen en dos etapas (*multi-stage build*) para minimizar el tamaño final [?].

Consecuencias: Mayor número de archivos de configuración; compensado por la independencia de despliegue y el menor tamaño de imagen.

ADR-017 — Variables de entorno para configuración sensible

Estado: Aceptado

Contexto: Incluir credenciales (contraseñas de base de datos, claves JWT) directamente en el código fuente o en archivos `.properties` versionados expone información sensible en el repositorio [?].

Decisión: Todas las configuraciones sensibles se pasan mediante variables de entorno definidas en el archivo `.env` (excluido de Git con `.gitignore`) y referenciadas en `docker-compose.yml`. En producción se usarán Docker secrets para las credenciales más críticas [?].

Consecuencias: El archivo `.env` debe compartirse de forma segura entre el equipo; se documenta el procedimiento en el README.

ADR-018 — Volúmenes Docker para persistencia de datos

Estado: Aceptado

Contexto: Sin persistencia explícita, los datos de PostgreSQL se pierden al recrear el contenedor. Se evaluaron volúmenes Docker, montajes de directorio (*bind mounts*) y bases de datos externas al contenedor [?].

Decisión: Se usan volúmenes Docker nombrados (`pgdata_main`, `pgdata_docente`) gestionados por Docker Compose. Los volúmenes sobreviven a la recreación de contenedores y permiten respaldar con `docker run -volumes-from`.

Consecuencias: Los datos quedan en el host; se recomienda respaldo diario automatizado mediante un contenedor de respaldo en el mismo Compose.

10.5 Backend

ADR-019 — Spring Boot 3 como framework backend

Estado: Aceptado

Contexto: El equipo evaluó Spring Boot 3 [?], Quarkus [?] y Micronaut [?] para el desarrollo de los microservicios.

Decisión: Se adopta Spring Boot 3 con Java 21. Spring Boot ofrece el ecosistema más maduro para microservicios empresariales, con integración nativa de Spring Security, Spring Data JPA y Spring Cloud Gateway. El equipo cuenta con experiencia previa en Spring, reduciendo la curva de aprendizaje [?].

Consecuencias: Mayor consumo de memoria en reposo frente a Quarkus (*native*); aceptable dado que la institución dispone de servidor con 8 GB RAM.

ADR-020 — Generación de reportes PDF con JasperReports

Estado: Aceptado

Contexto: El sistema debe generar boletas de calificaciones y actas en formato PDF descargable (RF26–RF29). Se evaluaron JasperReports [?], iText [?] y Apache PDFBox [?].

Decisión: Se adopta JasperReports 7 para la generación de reportes PDF. JasperReports permite diseñar plantillas visuales con Jaspersoft Studio, separando el diseño del código, y es compatible con Spring Boot mediante `jasperreports-spring-boot-starter`.

Consecuencias: Las plantillas `.jrxml` deben versionarse junto al código; el equipo debe familiarizarse con el diseñador visual.

ADR-021 — Versionado semántico de la API REST

Estado: Aceptado

Contexto: Sin versionado, un cambio en el contrato API rompe los clientes existentes. Se evaluaron versionado por URI (`/v1/`), por cabecera HTTP y por parámetro de consulta [?].

Decisión: Se adopta versionado por URI (`/api/v1/` y `/docente/v1/`), siguiendo la práctica recomendada por Massé [?]. Esta estrategia es la más explícita y compatible con herramientas de documentación (Swagger/OpenAPI).

Consecuencias: Futuros cambios incompatibles requerirán crear `/v2/`; los clientes de `/v1/` seguirán funcionando sin modificación.

10.6 Frontend

ADR-022 — Aplicación de página única (SPA) como interfaz web

Estado: Aceptado

Contexto: La interfaz debe ser responsiva (RNF09) y funcionar en navegadores modernos sin instalación local. Se evaluaron SPA (React/Angular), renderizado en servidor (SSR con Next.js) y plantillas HTML tradicionales (Thymeleaf) [?].

Decisión: Se adopta una SPA con React 18. El renderizado en cliente reduce la carga del servidor y permite una experiencia de usuario fluida para el registro de calificaciones (operación frecuente). Thymeleaf fue descartado por su acoplamiento con Spring y la menor experiencia de usuario [?].

Consecuencias: El primer cargado requiere descargar el *bundle* JS; se mitiga con *code splitting* y caché del navegador.

10.7 Calidad y proceso

ADR-023 — Git Flow como estrategia de ramificación

Estado: Aceptado

Contexto: Con cuatro integrantes trabajando en paralelo, se necesita una estrategia de ramificación que evite conflictos y permita mantener una rama estable para demostraciones [?].

Decisión: Se adopta Git Flow con ramas `main` (producción), `develop` (integración), `feature/*` (funcionalidades), `release/*` (preparación de entrega) y `hotfix/*` (correcciones urgentes) [?]. Los *pull requests* requieren revisión de al menos un integrante antes de fusionarse a `develop`.

Consecuencias: Mayor disciplina de proceso; compensado por la reducción de conflictos de integración en entregas.

ADR-024 — OpenAPI 3.0 para documentación del contrato REST

Estado: Aceptado

Contexto: Sin documentación formal del contrato API, la integración entre el frontend y el backend genera dependencias implícitas y errores difíciles de diagnosticar [?].

Decisión: Se adopta OpenAPI 3.0 con Swagger UI, generado automáticamente desde las anotaciones de Spring Boot (`springdoc-openapi`). La especificación se versiona en el repositorio y el entorno de desarrollo expone Swagger UI en `/swagger-ui.html` [?].

Consecuencias: Las anotaciones OpenAPI agregan verbosidad al código; compensado por la capacidad de generar clientes y pruebas automáticas desde la especificación.

ADR-025 — Convención de mensajes de commit semántico

Estado: Aceptado

Contexto: Sin convención de mensajes, el historial de Git es difícil de interpretar y no permite generar changelogs automáticos. La rúbrica del proyecto exige mensajes descriptivos y *pull requests* con revisión entre pares [?].

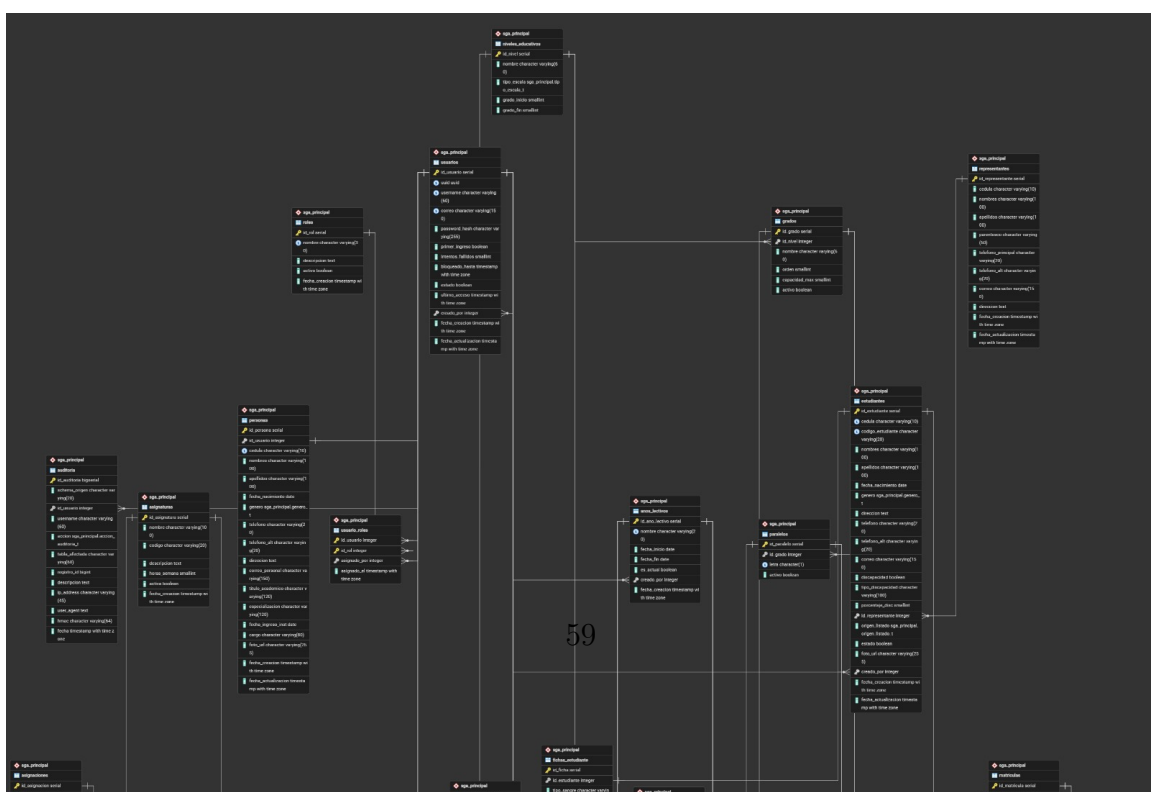
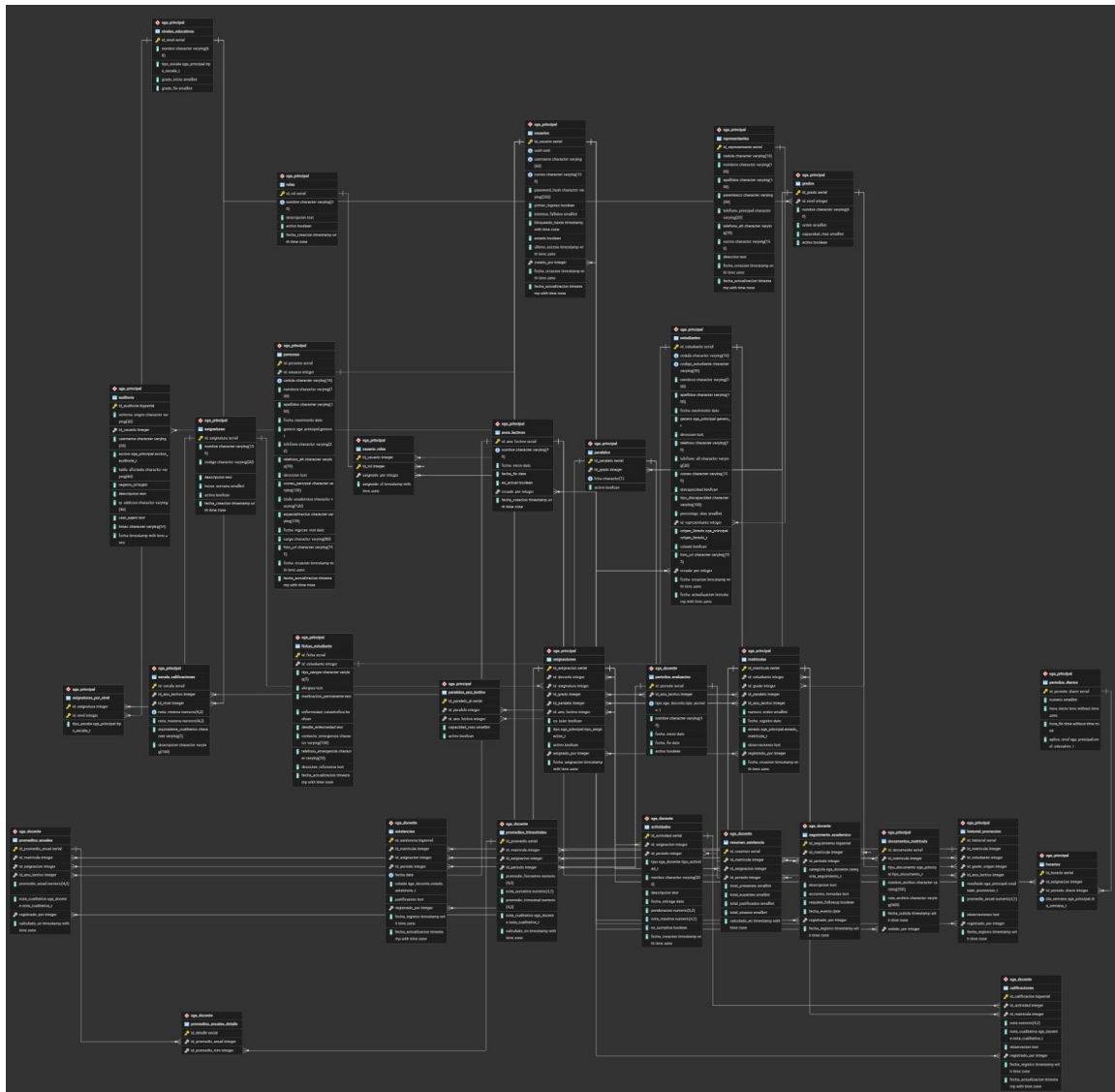
Decisión: Se adopta la especificación *Conventional Commits* [?]: `feat:`, `fix:`, `docs:`, `chore:`, `refactor:`, `test:`. Cada *commit* que cierra un requerimiento incluye la referencia al RF correspondiente (ej. `feat(matricula): implementar RF14 validación de duplicados`). Se requieren ≥ 3 *commits* por integrante por semana.

Consecuencias: Requiere disciplina del equipo; se automatiza la validación con `commitlint` como *pre-commit hook*.

11. Modelo de Datos Distribuido

La base de datos utiliza PostgreSQL 16 con dos schemas independientes que reflejan la separación de dominios de la arquitectura de microservicios [14].

11.1 Schema sga_main — Sistema Principal



11.2 Schema sga_docente — Microservicio Docente

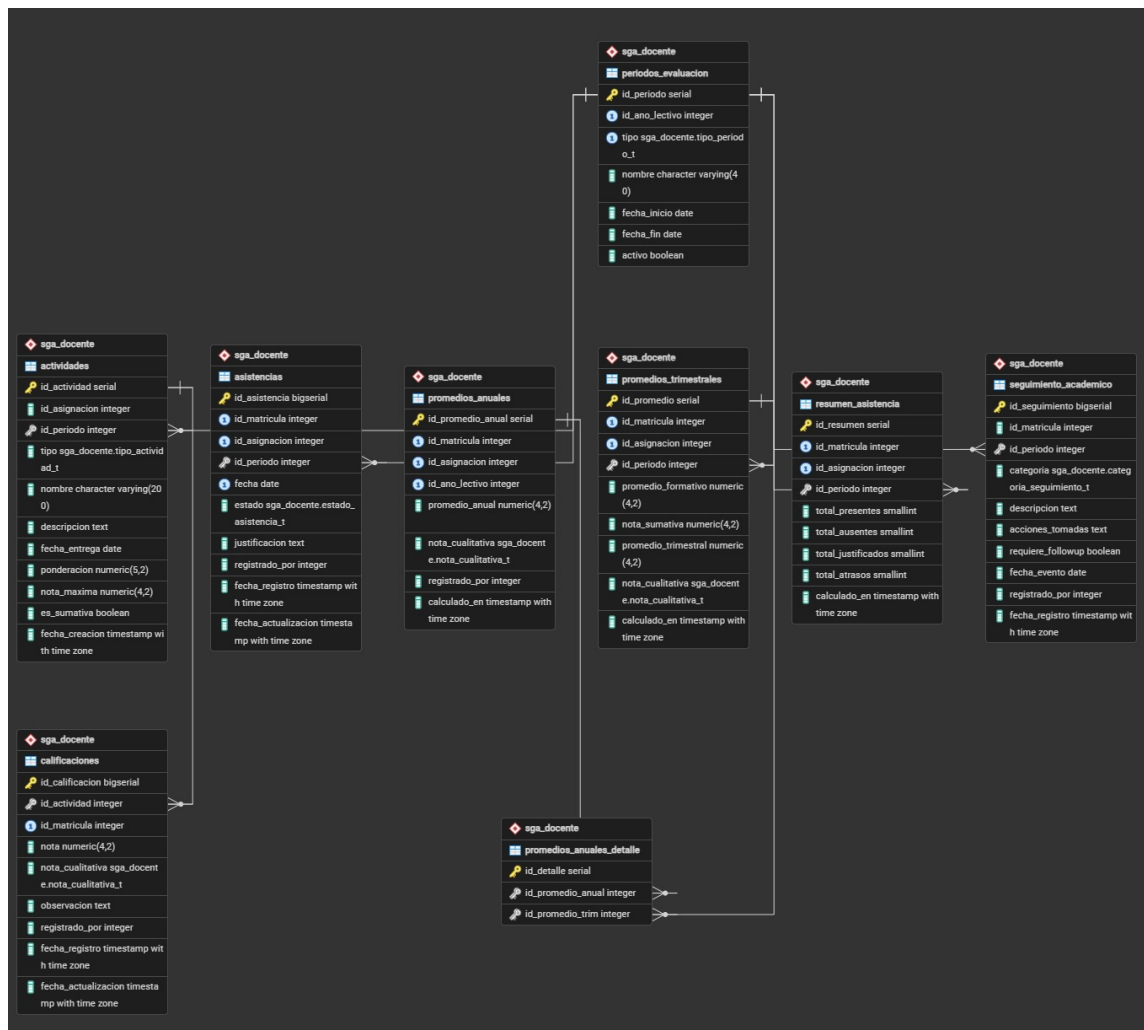


Figura 6: Diagrama ER — Schema sga_docente

Nota sobre referencias externas: Los campos marcados como *ref.ext.* son identificadores UUID que referencian entidades del schema `sga_main`. Por diseño de microservicios, no se usan foreign keys cruzadas entre schemas; la integridad referencial se garantiza a nivel de aplicación en el API Gateway.

12. Estrategia de Contenerización con Docker

La contenerización garantiza entornos reproducibles y facilita el despliegue en el servidor local de la institución [12, 20]. Se utiliza Docker Compose para orquestar los cuatro contenedores del sistema.

12.1 Dockerfile — Sistema Principal

Listing 1: Dockerfile del Sistema Principal (puerto 8080)

```
1 #           Etapa 1: Compilaci n
2 FROM eclipse-temurin:21-jdk-alpine AS builder
3 WORKDIR /app
4
5 # Copiar archivos de dependencias primero (cache de capas)
6 COPY mvnw pom.xml ./
7 COPY .mvn .mvn
8 RUN ./mvnw dependency:go-offline -q
9
10 # Compilar y empaquetar sin tests
11 COPY src ./src
12 RUN ./mvnw package -DskipTests -q
13
14 #           Etapa 2: Imagen de ejecuci n
15 FROM eclipse-temurin:21-jre-alpine AS runtime
16 WORKDIR /app
17
18 # Usuario no privilegiado para seguridad
19 RUN addgroup -S sga && adduser -S sga -G sga
20 USER sga
21
22 # Copiar JAR desde la etapa de compilaci n
23 COPY --from=builder /app/target/sga-main-*.jar app.jar
24
25 # Variables de entorno por defecto (sobreescritas en docker-compose
26   )
27 ENV SPRING_PROFILES_ACTIVE=prod
28 ENV SERVER_PORT=8080
29
30 EXPOSE 8080
31
32 # Health check
33 HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --
34   retries=3 \
35   CMD wget -qO- http://localhost:8080/actuator/health || exit 1
36
37 ENTRYPOINT ["java", "-Xmx512m", "-Xms256m", \
38   "-Djava.security.egd=file:/dev/./urandom", \
39   "-jar", "app.jar"]
```

12.2 Dockerfile — Microservicio Docente

Listing 2: Dockerfile del Microservicio Docente (puerto 8081)

```
1 #           Etapa 1: Compilaci n
2 FROM eclipse-temurin:21-jdk-alpine AS builder
3 WORKDIR /app
4
5 COPY mvnw pom.xml ./
6 COPY .mvn .mvn
7 RUN ./mvnw dependency:go-offline -q
8
9 COPY src ./src
10 RUN ./mvnw package -DskipTests -q
11
12 #           Etapa 2: Imagen de ejecuci n
13 FROM eclipse-temurin:21-jre-alpine AS runtime
14 WORKDIR /app
15
16 RUN addgroup -S sga && adduser -S sga -G sga
17 USER sga
18
19 COPY --from=builder /app/target/sga-docente-*.jar app.jar
20
21 ENV SPRING_PROFILES_ACTIVE=prod
22 ENV SERVER_PORT=8081
23
24 EXPOSE 8081
25
26 HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --
    retries=3 \
27     CMD wget -qO- http://localhost:8081/actuator/health || exit 1
28
29 ENTRYPOINT ["java", "-Xmx384m", "-Xms192m", \
30             "-Djava.security.egd=file:/dev/./urandom", \
31             "-jar", "app.jar"]
```

12.3 Dockerfile — Frontend React

Listing 3: Dockerfile del Frontend React (puerto 3000)

```
1 #           Etapa 1: Compilaci n
2 FROM node:20-alpine AS builder
```

```

3 WORKDIR /app
4
5 COPY package.json package-lock.json ./
6 RUN npm ci --silent
7
8 COPY . .
9 RUN npm run build
10
11 #           Etapa 2: Servidor Nginx
12
12 FROM nginx:1.25-alpine AS runtime
13
14 # Copiar build est tico
15 COPY --from=builder /app/dist /usr/share/nginx/html
16
17 # Configuraci n de nginx para SPA (react-router)
18 COPY nginx.conf /etc/nginx/conf.d/default.conf
19
20 EXPOSE 80
21
22 HEALTHCHECK --interval=30s --timeout=5s \
23   CMD wget -qO- http://localhost/ || exit 1
24
25 CMD ["nginx", "-g", "daemon off;"]

```

12.4 docker-compose.yml completo

Listing 4: docker-compose.yml — Orquestación completa del SGA

```

1 version: "3.9"
2
3 #           Redes
4
5 networks:
6   sga-backend:
7     driver: bridge
8   sga-frontend:
9     driver: bridge
10
11 #           Vol menes persistentes
12
13 volumes:
14   postgres-data:
15     driver: local

```



```
15 services:
16
17     #           Base de datos PostgreSQL
18
19     postgres:
20         image: postgres:16-alpine
21         container_name: sga-postgres
22         restart: unless-stopped
23         environment:
24             POSTGRES_USER: ${DB_USER:-sga_admin}
25             POSTGRES_PASSWORD: ${DB_PASSWORD}
26             POSTGRES_DB: sga
27         volumes:
28             - postgres-data:/var/lib/postgresql/data
29             - ./init-db:/docker-entrypoint-initdb.d:ro
30         networks:
31             - sga-backend
32         healthcheck:
33             test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-sga_admin} -d sga"]
34             interval: 15s
35             timeout: 5s
36             retries: 5
37
38     #           Sistema Principal
39
40     sga-main:
41         build:
42             context: ./sga-main
43             dockerfile: Dockerfile
44         container_name: sga-main
45         restart: unless-stopped
46         depends_on:
47             postgres:
48                 condition: service_healthy
49         environment:
50             SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/sga?currentSchema=sga_main
51             SPRING_DATASOURCE_USERNAME: ${DB_USER:-sga_admin}
52             SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
53             JWT_SECRET: ${JWT_SECRET}
54             JWT_EXPIRATION_MS: 28800000
55             BCrypt_STRENGTH: 12
56             AUDIT_HMAC_KEY: ${HMAC_KEY}
57         networks:
58             - sga-backend
59         expose:
```

```

58     - "8080"
59   healthcheck:
60     test: ["CMD", "wget", "-q0-", "http://localhost:8080/actuator
61       /health"]
62     interval: 30s
63     timeout: 10s
64     start_period: 60s
65     retries: 3
66 #           Microservicio Docente

67 sga-docente:
68   build:
69     context: ./sga-docente
70     dockerfile: Dockerfile
71   container_name: sga-docente
72   restart: unless-stopped
73   depends_on:
74     postgres:
75       condition: service_healthy
76     sga-main:
77       condition: service_healthy
78   environment:
79     SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/sga?
80       currentSchema=sga_docente
81     SPRING_DATASOURCE_USERNAME: ${DB_DOCENTE_USER:-sga_docente}
82     SPRING_DATASOURCE_PASSWORD: ${DB_DOCENTE_PASSWORD}
83     JWT_SECRET: ${JWT_SECRET}
84     SGA_MAIN_URL: http://sga-main:8080
85     AUDIT_HMAC_KEY: ${HMAC_KEY}
86   networks:
87     - sga-backend
88   expose:
89     - "8081"
90   healthcheck:
91     test: ["CMD", "wget", "-q0-", "http://localhost:8081/actuator
92       /health"]
93     interval: 30s
94     timeout: 10s
95     start_period: 60s
96     retries: 3
97 #           API Gateway

98 api-gateway:
99   build:
100     context: ./api-gateway

```

```

100     dockerfile: Dockerfile
101     container_name: sga-gateway
102     restart: unless-stopped
103     depends_on:
104         sga-main:
105             condition: service_healthy
106         sga-docente:
107             condition: service_healthy
108     environment:
109         SGA_MAIN_URL: http://sga-main:8080
110         SGA_DOCENTE_URL: http://sga-docente:8081
111         JWT_SECRET: ${JWT_SECRET}
112         RATE_LIMIT_RPM: 100
113     networks:
114         - sga-backend
115         - sga-frontend
116     ports:
117         - "8443:8443"
118
119 #           Frontend React
120
121 sga-frontend:
122     build:
123         context: ./sga-frontend
124         dockerfile: Dockerfile
125     container_name: sga-frontend
126     restart: unless-stopped
127     depends_on:
128         - api-gateway
129     environment:
130         VITE_API_BASE_URL: https://api-gateway:8443
131     networks:
132         - sga-frontend
133     ports:
134         - "3000:80"

```

12.5 Variables de entorno — archivo .env

Listing 5: Ejemplo de archivo .env (nunca versionar en Git)

```

1 # Base de datos - Sistema Principal
2 DB_USER=sga_admin
3 DB_PASSWORD=C4mbi4M3_5egur0!
4
5 # Base de datos - Microservicio Docente
6 DB_DOCENTE_USER=sga_docente
7 DB_DOCENTE_PASSWORD=D0c3nt3_5egur0!

```

```

8
9 # JWT (m nimo 64 caracteres aleatorios)
10 JWT_SECRET=
    tU5ecr3t0_d3_64_car4ct3res_m1n1m0_par4_HMAC_SHA256_aqui_va_la_clave
11
12 # HMAC para logs de auditoria
13 HMAC_KEY=0
    tr4_cl4v3_64_char_par4_HMAC_audit_logs_SGA_ProvinciasUnidas_2026

```

12.6 Resumen de la estrategia de contenedores

Contenedor	Puerto externo	Red	Imagen base
sga-postgres	—	sga-backend	postgres:16-alpine
sga-main	—	sga-backend	eclipse-temurin:21-jre-alpine
sga-docente	—	sga-backend	eclipse-temurin:21-jre-alpine
sga-gateway	8443	backend+front	eclipse-temurin:21-jre-alpine
sga-frontend	3000	sga-frontend	nginx:1.25-alpine

Cuadro 17: Resumen de contenedores y configuración de red

13. Protocolo Experimental Reproducible

13.1 Descripción en español

Este protocolo describe los pasos necesarios para levantar el Sistema de Gestión Académica desde cero en un entorno local, con el objetivo de garantizar la reproducibilidad del experimento en un tiempo menor a 30 minutos.

13.1.1. Requisitos del entorno

Componente	Versión / Especificación
Sistema operativo	Ubuntu 22.04 LTS (64-bit) o Windows 11 con WSL2
Docker Engine	25.0 o superior (<code>docker --version</code>)
Docker Compose	Plugin v2.24 o superior (<code>docker compose version</code>)
RAM mínima	4 GB (recomendado: 8 GB)
Espacio en disco	5 GB libres para imágenes y volúmenes
Conexión a internet	Requerida solo para la descarga inicial de imágenes base
Puertos disponibles	3000 (frontend), 8443 (API Gateway)
Git	2.x o superior

Cuadro 18: Requisitos del entorno para despliegue

13.1.2. Pasos de despliegue

1. Clonar el repositorio del proyecto:

```
1 git clone https://github.com/equipo-bcel/sga-provincias-unidas
  .git
2 cd sga-provincias-unidas
```

2. Crear el archivo de variables de entorno:

```
1 cp .env.example .env
2 # Editar .env y establecer contraseñas seguras
3 nano .env
```

3. Construir las imágenes Docker:

```
1 docker compose build --no-cache
2 # Tiempo estimado: 8-12 minutos (primera vez)
```

4. Inicializar la base de datos:

```
1 # Los scripts en ./init-db/ crean los schemas y el usuario
  inicial
2 # Se ejecutan automáticamente al primer arranque de postgres
3 docker compose up postgres -d
4 docker compose logs -f postgres # Esperar "ready to accept
  connections"
```

5. Levantar todos los servicios:

```
1 docker compose up -d
2 # Verificar estado de todos los contenedores
3 docker compose ps
```

6. Verificar el estado de salud de los servicios:

```

1 # Esperar que todos los contenedores reporten "healthy"
2 docker compose ps --format "table {{.Name}}\t{{.Status}}"
3
4 # Verificar API Gateway
5 curl -k https://localhost:8443/actuator/health
6
7 # Verificar Sistema Principal
8 curl http://localhost:8080/actuator/health
9
10 # Verificar Microservicio Docente
11 curl http://localhost:8081/actuator/health

```

7. Acceder al sistema:

Abrir el navegador en <http://localhost:3000>. Usar las credenciales del administrador inicial:

- Usuario: `admin@provinciasunidas.edu.ec`
- Contraseña: `Admin1234!` (el sistema solicitará cambio inmediato)

8. Detener el sistema:

```

1 docker compose down          # Detiene contenedores (conserva
    datos)
2 docker compose down -v      # Detiene y elimina volúmenes (
    datos borrados)

```

13.1.3. Resolución de problemas frecuentes

Problema	Solución
Puerto 3000 ocupado	<code>sudo lsof -i:3000</code> para identificar el proceso y terminarlo.
Contenedor en estado <i>unhealthy</i>	<code>docker compose logs <nombre></code> para ver el error específico.
Error de memoria insuficiente	Reducir <code>-Xmx</code> en los Dockerfiles o liberar RAM cerrando otras aplicaciones.
Base de datos no inicializada	<code>docker compose down -v && docker compose up -d</code> para recrear volúmenes.

Cuadro 19: Problemas frecuentes y soluciones

13.2 Reproducible Experimental Protocol (English)

Title: Deployment and verification protocol for the Academic Management System (SGA) — Escuela «Provincias Unidas».

Objective: To enable any evaluator or developer to reproduce the complete SGA environment on a local machine within 30 minutes, starting from the project repository.

Environment requirements: Ubuntu 22.04 LTS (or Windows 11 with WSL2), Docker Engine 25.0+, Docker Compose Plugin v2.24+, minimum 4 GB RAM, 5 GB free disk space, and ports 3000 and 8443 available.

Procedure:

1. Clone the repository: `git clone {repository-url} && cd sga-provincias-unidas`
2. Configure environment variables: copy `.env.example` to `.env` and set secure passwords for the database and JWT secret.
3. Build Docker images: `docker compose build --no-cache` (estimated time: 8–12 minutes on first run).
4. Start all services: `docker compose up -d`. Docker Compose will automatically start services in the correct dependency order (PostgreSQL → sga-main → sga-docente → api-gateway → sga-frontend).
5. Wait for all containers to report *healthy* status: `docker compose ps`. Typical startup time is 90–120 seconds after image build.
6. Verify system availability by navigating to `http://localhost:3000` in any modern web browser.
7. Log in with the initial administrator credentials (username: `admin@provinciasunidas.edu.ec`, password: `Admin1234!`). The system will immediately prompt for a mandatory password change.
8. To validate the REST API directly, send a POST request to `https://localhost:8443/api/v1/auth/login` with the JSON body `{ "correo": "admin@...", "password": "new_password" }` and verify that the response includes a valid JWT token.
9. To stop all services without losing data: `docker compose down`. To also remove persistent volumes: `docker compose down -v`.

Expected results: All five containers (postgres, sga-main, sga-docente, api-gateway, sga-frontend) report *healthy* status. The login endpoint returns HTTP 200 with a JWT token. The frontend renders the login screen without console errors. The entire process completes in under 30 minutes on hardware meeting the minimum specifications.

Reproducibility guarantee: The multi-stage Docker builds ensure identical runtime environments regardless of the host operating system. The `.env` file is the only host-specific configuration required.

Referencias

- [1] Asamblea Nacional del Ecuador. Ley orgánica de educación intercultural (LOEI). Registro Oficial Suplemento N.º 417, 2011. Disponible en: <https://educacion.gob.ec/wp-content/uploads/downloads/2017/05/Ley-Organica-Educacion-Intercultural-Codificado.pdf>.
- [2] Paolo Di Francesco, Ivano Malavolta, and Patricia Lago. Research on architecting microservices: Trends, focus, and potential for industrial adoption. In *Proceedings of the IEEE International Conference on Software Architecture (ICSA)*, pages 21–30. IEEE, 2017. doi: 10.1109/ICSA.2017.11.
- [3] Martin Fowler and James Lewis. Microservices: A definition of this new architectural term. <https://martinfowler.com/articles/microservices.html>, 2014. Accedido: enero de 2025.
- [4] Chris Richardson. *Microservices Patterns: With Examples in Java*. Manning Publications, Shelter Island, NY, 2018. ISBN 978-1-61729-454-9. URL <https://www.manning.com/books/microservices-patterns>.
- [5] Ian Sommerville. *Software Engineering*. Pearson Education, Hoboken, NJ, 10 edition, 2016. ISBN 978-0-13-394303-0. URL <https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003258>.
- [6] Bashar Nuseibeh and Steve Easterbrook. Requirements engineering: A roadmap. In *Proceedings of the Conference on the Future of Software Engineering (ICSE 2000)*, pages 35–46. ACM, 2000. doi: 10.1145/336512.336523.
- [7] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, Boston, MA, 4 edition, 2021. ISBN 978-0-13-688609-9. URL <https://www.informit.com/store/software-architecture-in-practice-9780136886099>.
- [8] Niels Provos and David Mazières. A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference (FREENIX Track)*, pages 81–91, Monterey, CA, 1999. USENIX Association. URL <https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf>.
- [9] David F. Ferraiolo, Ravi Sandhu, Serban Gavrila, D. Richard Kuhn, and Ramaswamy Chandramouli. Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security*, 4(3):224–274, 2001. doi: 10.1145/501978.501980.
- [10] Michael Jones, John Bradley, and Nat Sakimura. RFC 7519: JSON web token (JWT). Technical report, Internet Engineering Task Force (IETF), 2015. URL <https://www.rfc-editor.org/rfc/rfc7519>.
- [11] Eric Rescorla. RFC 8446: The transport layer security (TLS) protocol version 1.3. Technical report, Internet Engineering Task Force (IETF), 2018. URL <https://www.rfc-editor.org/rfc/rfc8446>.

- [12] Carl Boettiger. An introduction to Docker for reproducible research, with examples from the R environment. *ACM SIGOPS Operating Systems Review*, 49(1):71–79, 2015. doi: 10.1145/2723872.2723882.
- [13] Roy Thomas Fielding. *Architectural Styles and the Design of Network-Based Software Architectures*. PhD thesis, University of California, Irvine, 2000. URL <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [14] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, Sebastopol, CA, 2017. ISBN 978-1-4493-7332-0. URL <https://dataintensive.net>.
- [15] Claus Pahl, Antonio Brogi, Jacopo Soldani, and Pooyan Jamshidi. Cloud container technologies: A state-of-the-art review. *IEEE Transactions on Cloud Computing*, 7(3):677–692, 2017. doi: 10.1109/TCC.2017.2702586.
- [16] Presidencia de la República del Ecuador. Reglamento general a la LOEI. Decreto Ejecutivo N.º 1241, R.O. Suplemento N.º 754, 2012. Disponible en: <https://educacion.gob.ec/wp-content/uploads/downloads/2023/05/RL0EI.pdf>.
- [17] Asamblea Nacional del Ecuador. Ley orgánica de protección de datos personales. Registro Oficial Suplemento N.º 459, 2021. Disponible en: <https://www.telecomunicaciones.gob.ec/wp-content/uploads/2021/06/Ley-Organica-de-Datos-Personales.pdf>.
- [18] Nuha Alshuqayran, Nour Ali, and Roger Evans. A systematic mapping study in microservice architecture. In *Proceedings of the 9th IEEE International Conference on Service-Oriented Computing and Applications (SOCA)*, pages 44–51. IEEE, 2016. doi: 10.1109/SOCA.2016.15.
- [19] Armin Balalaie, Abbas Heydarnoori, and Pooyan Jamshidi. Microservices architecture enables DevOps: Migration to a cloud-native architecture. *IEEE Software*, 33(3):42–52, 2016. doi: 10.1109/MS.2016.64.
- [20] David Bernstein. Containers and cloud: From LXC to Docker to Kubernetes. *IEEE Cloud Computing*, 1(3):81–84, 2014. doi: 10.1109/MCC.2014.51.